Architettura degli elaboratori e sistemi operativi

Diego Stefanini — prof. Marco Danelutto, a.a. 2026-27

Indice

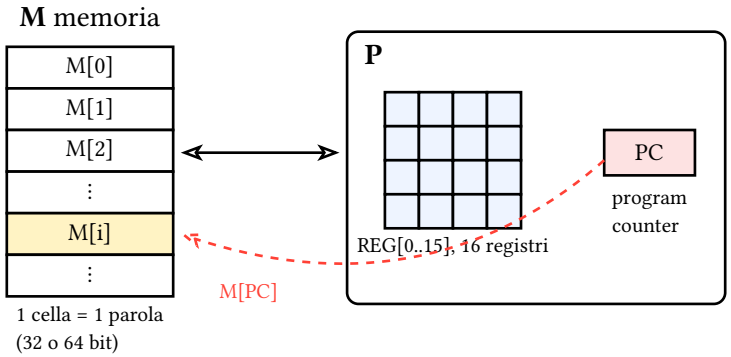
1 Fondamenti dei sistemi di elaborazione 3
1.1 Memoria, processore, istruzioni 3
1.2 Astrazione 4
1.3 Numeri binari 5
1.4 Basi, numeri reali e caratteri 9
2 Reti logiche 11
2.1 Porte logiche e tabelle di verità 11
2.2 Tempo di stabilizzazione 13
2.3 Componenti 14
2.4 Semplificare le formule 17
2.5 Descrivere le reti in Verilog 21
2.6 Memoria: latch, flip-flop, registro 27
2.7 Reti sequenziali 29
2.8 Reti sequenziali coi componenti e in Verilog 32

▸ nel margine: cliccando si apre la registrazione della lezione (serve l'account Unipi) accanto c'è il numero del video, poi il minuto da cui parte la spiegazione; sotto, fin dove ascoltare.

1 Fondamenti dei sistemi di elaborazione

1.1 Memoria, processore, istruzioni

Un calcolatore è fatto di due parti che si scambiano dati di continuo: la **memoria** M, che contiene programmi e dati, e il **processore** P, che esegue i programmi.



Il PC contiene l'indirizzo della prossima istruzione da prendere in memoria

- La **memoria** è un lungo vettore di celle numerate. Il numero di una cella è il suo **indirizzo**: M[i] è la cella di indirizzo i. Ogni cella contiene una **parola** di 32 o 64 bit (un bit è una cifra 0 o 1, vedi i numeri binari più avanti).
- I **registri** sono poche celle (qui 16) **dentro** il processore: molto più veloci della memoria, ci stanno i dati su cui il processore sta lavorando in quel momento.
- Il **PC** (*program counter*) è un registro speciale: contiene l'indirizzo della prossima istruzione da eseguire. Anche il programma, infatti, sta in memoria.

Una volta avviato, il processore ripete per sempre questi quattro passi, il **ciclo fetch-decode-execute**:

```
while (true) {
    preleva l'istruzione da M[PC]
    decodifica      // che istruzione è?
    esegue
    scrive i risultati
}
```

Dopo il prelievo il PC passa all'istruzione successiva, così al giro dopo il processore prende quella dopo.

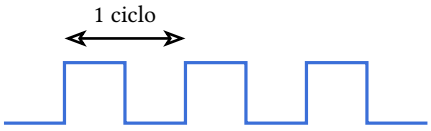
Quali istruzioni esistono dipende dal processore (x86, ARM, RISC-V sono famiglie di processori diverse), ma i tipi sono sempre tre:

Tipo	Istruzioni	Esempio
operative	`ADD SUB MUL DIV` `AND OR NOT` `SHIFT ROT` (sx/dx)	`ADD R0, R1, R2` ⟶ REG[0] = REG[1] + REG[2] `SHIFT` sposta i bit a sinistra o a destra; `ROT` li ruota: il bit che esce da una parte rientra dall'altra
memoria	`LOAD STORE`	`LOAD R5, addr` ⟶ REG[5] = M[addr] (addr = un indirizzo) `STORE` fa il contrario: registro ⟶ memoria
salto	salto	cambia il PC: la prossima istruzione non è quella dopo

```
inizio:  ADD R0, R0, R0   ← PC
         ADD R1, R1, R1
         SUB R2, R0, R1
         salto inizio     ┐
            ↑─────────────┘
```

Il PC scende di un'istruzione alla volta; il **salto** lo rimette su `inizio`, quindi il programma ricomincia.

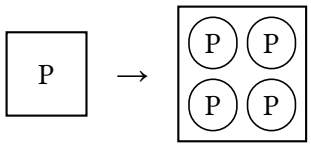
A dare il ritmo ai passi del ciclo è il **clock**, un segnale che passa da 0 a 1 e viceversa a intervalli regolari. Ogni passo del processore dura un certo numero di cicli di clock: più cicli al secondo (la **frequenza**, in Hz), più istruzioni al secondo.



clock a 1 GHz ⟶ 1 ciclo = 1 ns

Il clock scandisce i passi del processore

- 1 GHz = 10^9 cicli al secondo, quindi un ciclo dura 10^{-9} s = 1 ns.
- Il processore è fatto di **transistor**, minuscoli interruttori. Nel Motorola 68000 (fine anni "70) misuravano 20-40 µm, oggi 2 nm *(1000 nm = 1 µm)*.
- Transistor più piccoli ⟶ nello spazio di un processore ne stanno più di uno. Ogni processore sul chip si chiama **core**, il chip è **multicore** (sotto: un chip con un solo processore e uno con quattro core):



> **Da sapere — legge di Moore** (aggiunto, non spiegato in aula)
> Il numero di transistor che si riescono a mettere su un chip raddoppia circa ogni due anni. È per questo che si passa dai µm ai nm e da un processore a più processori sullo stesso chip.

1.2 Astrazione

Un calcolatore si studia a **livelli di astrazione**: ognuno usa quello sotto senza doverne conoscere i dettagli.

Applicazioni
Sistema operativo — processi, memoria, file system
Architettura — set di istruzioni (ARMv7)
Microarchitettura — il processore
Logica — componenti: AND, OR, NOT, «memoria»
Circuiti digitali — segnali «1» e «0»: FA, MUX
Circuiti analogici
Dispositivi
Fisica — questi ultimi 3 non li facciamo

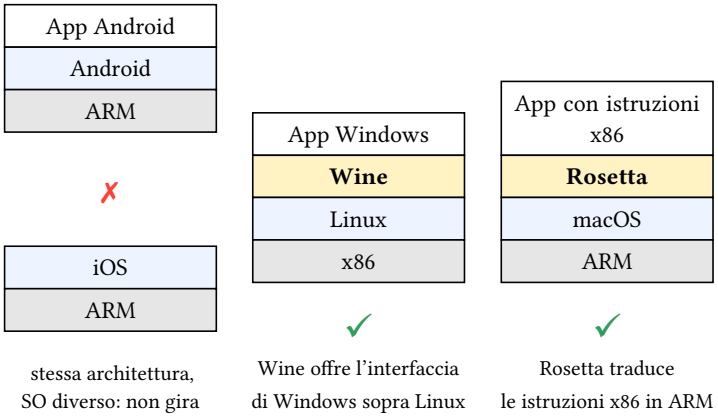
Il corso parte dal basso e risale. FA e MUX sono due circuiti che arrivano più avanti: il **full adder** somma una cifra, il **multiplexer** sceglie fra due segnali.

Applicazione — livello i+1
interfaccia: chiamate al sistema operativo
Sistema operativo — livello i
interfaccia: istruzioni in linguaggio macchina
Architettura — livello i−1

Fra due livelli c'è un'**interfaccia**: l'insieme delle funzionalità che il livello i−1 mette a disposizione del livello i.

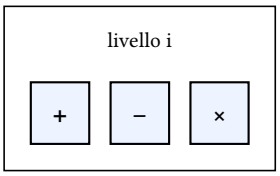
Il livello i usa solo l'interfaccia, non sa com'è fatto sotto.

Conseguenza: un programma gira solo dove trova l'interfaccia per cui è stato scritto. Un'app Android usa le chiamate di sistema di Android: su iOS non ci sono, anche se il processore (ARM) è lo stesso. Al contrario, basta che qualcuno offra l'interfaccia giusta, anche «finta», e il programma gira:

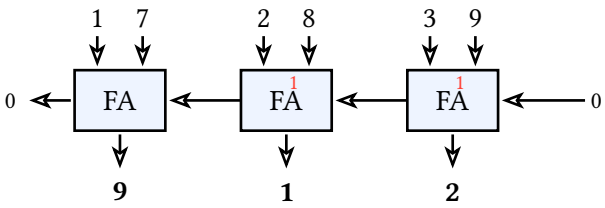


Per costruire sistemi così complessi si usano tre principi:

- **Gerarchia**: il sistema è diviso in livelli (sopra).
- **Modularità**: ogni livello è fatto di moduli con un compito preciso.
- **Regolarità**: lo stesso modulo si riusa tante volte.



Modularità: un livello fatto di moduli

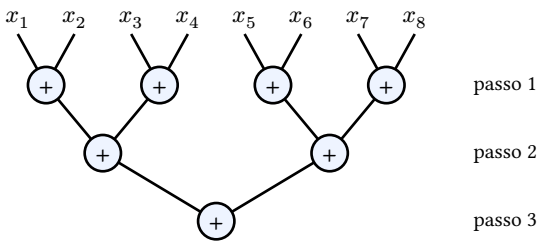


Regolarità: 123 + 789 = 912 con tre FA uguali in cascata. Ogni FA somma una cifra e passa il riporto (in rosso) a sinistra

Stesso modulo `+`, due modi di sommare 8 numeri:

```
sum = 0;
for (i = 0; i < n; i++)
    sum = sum + x[i];
```

7 somme **una dopo l'altra**: ognuna aspetta il risultato della precedente, quindi 7 passi.



Ad albero: le somme dello stesso livello sono indipendenti, quindi si possono fare insieme (con 4 sommatori): le stesse 7 somme in 3 passi

1.3 Numeri binari

I circuiti digitali lavorano in **binario**: ogni segnale vale 0 o 1.

In un sistema posizionale **conta l'ordine delle cifre**: la stessa cifra vale di più quanto più è a sinistra. Ogni posizione ha un peso.

decimale	$10^3 = 1000$	$10^2 = 100$	$10^1 = 10$	$10^0 = 1$
binario	$2^3 = 8$	$2^2 = 4$	$2^1 = 2$	$2^0 = 1$

Esempio: $13_{10} \neq 31_{10}$ e $10_2 \neq 01_2$ (il numero piccolo in basso è la base).

Per passare **da binario a decimale** sommo i pesi delle posizioni dove c'è 1: $1100_2 = 8 + 4 = 12$.

Per passare **da decimale a binario** divido per 2 finché arrivo a 1, poi leggo i resti **dal basso verso l'alto**. Il resto della divisione per 2 dice se il numero è pari o dispari, cioè l'ultima cifra binaria; dividendo, «tolgo» quella cifra e passo alla successiva. Per questo le cifre escono da destra a sinistra.

n	**: 2**	**resto**
12	6	0
6	3	0
3	1	1
1		1 ↑

$12 = 1100_2$

n	**: 2**	**resto**
7	3	1
3	1	1
1		1 ↑

$7 = 111_2$

Somma e prodotto in binario si fanno come in decimale, ma $1 + 1 = 10_2$: scrivo 0 e riporto 1.

```
  8421          111
  1010          0111
+ 0100        + 0001
------        ------
  1110          1000
```

10 + 4 = 14 (pesi in grigio) — 7 + 1 = 8 (riporti in grigio)

Nel prodotto in colonna la cifra del moltiplicatore è 0 o 1, quindi ogni riga è tutta zeri oppure il numero stesso, spostata di un posto a sinistra (il · segna il posto lasciato vuoto).

```
   12            1010
× 34          × 0100
-----         ------
   48            0000
  36·           0000·
-----         1010··
  408         0000···
              -------
              0101000
```

in decimale — 10 × 4 = 40 = 32 + 8

Per i **numeri negativi** il modo più semplice è **modulo e segno**: il primo bit è il **segno** (0 = +, 1 = −), gli altri sono il **valore assoluto**.

1	**1**	**1**	**0**	**0**

segno — valore assoluto = 12

−12 in modulo e segno

Per sommare A e B bisogna prima guardare i segni (S = segno, V = valore assoluto):

Caso	Risultato
$S_A = S_B$	segno S_A, valore $V_A + V_B$
$S_A \neq S_B$ e $V_A > V_B$	segno S_A, valore $V_A - V_B$
altrimenti	segno S_B, valore $V_B - V_A$

Esempi: $+12 + 12 = +24$, $-12 + (-12) = -24$, $-12 + 7 = -5$ (segni diversi: faccio $12 - 7$ e tengo il segno del più grande).

> Servono confronti e sottrazioni: il circuito che somma diventa complicato. Il complemento a 2 risolve questo problema.

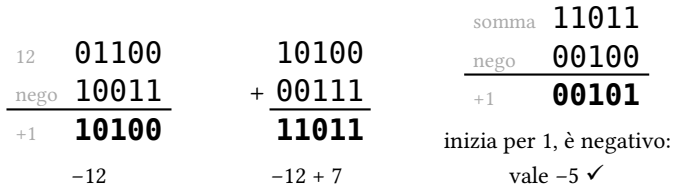
Si usa quindi il **complemento a 2**: i positivi si scrivono come sempre, e per passare da n a $-n$ si negano tutti i bit ($0 \leftrightarrow 1$) e si somma 1:

$$\boxed{n \text{ in binario}} \rightarrow \boxed{\text{nego tutti i bit}} \rightarrow \boxed{+1} \rightarrow \boxed{-n}$$

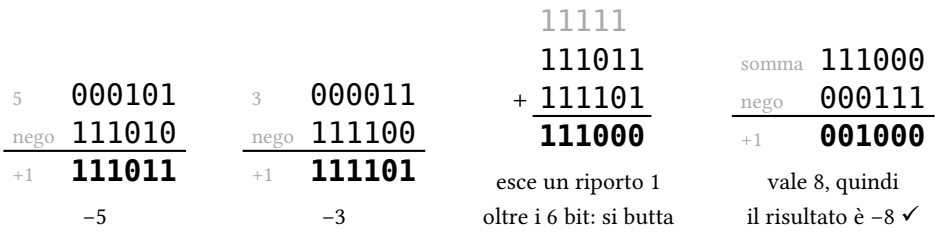
Vale anche al contrario: rifacendo «nego + 1» su $-n$ torno a n. Così, se un risultato comincia con 1 (è negativo), capisco quanto vale.

Il vantaggio: **la somma si fa come una somma normale**, senza guardare i segni.

Esempio: $-12 + 7$ su 5 bit.



Esempio: $-5 + (-3)$ su 6 bit.



> Nell'esempio −5 + (−3) esce un riporto oltre i 6 bit e il risultato è giusto lo stesso: **il riporto in uscita non è un errore**. L'errore vero è l'overflow, qui sotto.

Quanti numeri ci stanno in n bit? Ogni bit raddoppia le combinazioni: $2 \cdot 2 \cdot 2 \cdots = 2^n$. Con 8 bit sono $2^8 = 256$ combinazioni, e il bit in posizione i (contando da destra, da 0) pesa 2^i.

2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0

8 bit: sopra ogni casella il suo peso

binario	valore
0000 0000	0
0000 0001	1
0000 0010	2
0000 0011	3
0000 0100	4
⋮	⋮
1111 1111	255

Le 256 combinazioni, dalla più piccola alla più grande

Le 256 combinazioni si possono usare in due modi:

con n bit	intervallo	8 bit
interi **senza segno** (solo +)	$0 \ldots 2^n - 1$	$0 \ldots 255$
relativi (+ e −) in complemento a 2	$-2^{n-1} \ldots +2^{n-1} - 1$	$-128 \ldots +127$

In complemento a 2 metà delle combinazioni va ai negativi e metà a zero e positivi: per questo c'è un negativo in più (-128) e i positivi si fermano a $+127$.

Un numero in complemento a 2 si legge così: **il bit più a sinistra pesa -2^{n-1}**, gli altri pesano $+2^i$ come sempre. Il segno è già dentro i pesi: per questo basta la somma normale.

-128	2^6	2^5	2^4	2^3	2^2	2^1	2^0

$$1111\ 1101 = -128 + 64 + 32 + 16 + 8 + 4 + 1 = -3$$

valore	8 bit
3	0000 0011
2	0000 0010
1	0000 0001
0	0000 0000
−1	1111 1111
−2	1111 1110
−3	1111 1101
⋮	⋮
−128	1000 0000

- Tutti i negativi cominciano per 1, zero e positivi per 0.
- -1 è tutti 1: 0000 0001 ⟶ nego 1111 1110 ⟶ $+1$ ⟶ 1111 1111.
- -128 non ha l'opposto: facendo «nego + 1» torna se stesso, perché $+128$ non ci sta in 8 bit.

```
-128  1000 0000
nego  0111 1111
+1    1000 0000
```

> In **modulo e segno** l'intervallo è $-(2^{n-1}-1) \ldots +(2^{n-1}-1)$ e lo zero ha **due** rappresentazioni, $+0 = 0000\ 0000$ e $-0 = 1000\ 0000$. Anche un semplice `if (x == 0)` deve controllarle tutte e due: un altro motivo per cui si usa il complemento a 2.

Sommando due numeri di n bit il risultato può aver bisogno di $n+1$ bit, ma il circuito ne ha solo n. Quando il risultato esce dall'intervallo rappresentabile si ha **overflow** (traboccamento).

Esempi su 3 bit, dove il complemento a 2 va da -4 a $+3$:

−4	−3	−2	−1	0	1	2	3
100	101	110	111	000	001	010	011

```
  1
  010
+ 010
  100
```

2 + 2 = 4, ma 100 vale −4 ✗

```
   101
+  110
  (1)011
```

−3 + (−2) = −5, ma 011 vale +3 ✗

Regola: operandi con lo **stesso segno** e risultato con **segno diverso** ⟹ **overflow**.

Con segni diversi l'overflow non può mai capitare: il risultato sta sempre fra i due operandi.

Il complemento a 2 permette di fare anche la **sottrazione con lo stesso sommatore**: $a - b = a + (-b)$. Il circuito è fra i componenti delle reti logiche.

1.4 Basi, numeri reali e caratteri

Un sistema posizionale in **base B** usa le cifre da 0 a $B-1$, e la cifra in posizione i pesa B^i.

base	cifre	nota
$B = 10$	$0, 1, \ldots, 9$	
$B = 2$	$0, 1$	
$B = 8$	$0, 1, \ldots, 7$	una cifra = 3 bit
$B = 16$	$0, \ldots, 9, A, B, C, D, E, F$	una cifra = 4 bit (**nibble**); $A = 10, \ldots, F = 15$

In base 16 i pesi sono $16^2 = 256$, $16^1 = 16$, $16^0 = 1$. L'esadecimale si usa perché da binario si passa **senza conti**: $16 = 2^4$, quindi una cifra esadecimale corrisponde esattamente a 4 bit. Divido i bit in gruppi di 4 partendo da destra e scrivo ogni gruppo come una cifra. Il prefisso `0x` indica un numero esadecimale.

```
      6         D
  ┌─────┐   ┌─────┐
  0 1 1 0   1 1 0 1
128 64 32 16  8 4 2 1
```

$0110\ 1101 = \texttt{0x6D} = 64 + 32 + 8 + 4 + 1 = 109$

Esempio: -3 su 16 bit.

```
3     0000 0000 0000 0011
nego  1111 1111 1111 1100
+1    1111 1111 1111 1101
```

ogni gruppo 1111 è una F, 1101 è D:
$-3 = \texttt{0xFFFD}$

Per i **numeri con la virgola** la notazione posizionale continua a destra della virgola con esponenti negativi:

$$123{,}45 = 1 \times 10^2 + 2 \times 10^1 + 3 \times 10^0 + 4 \times 10^{-1} + 5 \times 10^{-2}$$

Tenendo la virgola in una posizione fissa, per sommare $123{,}45 + 17{,}9$ bisogna prima allinearla (017,90). Si usa invece la **virgola mobile** (**floating point**): cifre ed esponente separati, come nella notazione `mmmEn` = mmm $\times 10^n$.

`12345E-2` $= 123{,}45$ (la virgola si sposta di 2 a sinistra)
`123E2` $= 12300$

Il numero si scrive come $\pm$ 0,mantissa $\times 10^E$ e si salvano tre campi, in 32 o 64 bit:

S	E	mantissa
segno	esponente	valore 0,xxxx

$123{,}45 = +0{,}12345 \times 10^3$
S = 0, E = 3, mantissa = 12345

L'esponente può essere negativo: con 8 bit ha 256 valori, da -128 a $+127$. Così con pochi bit si scrivono numeri molto grandi e molto piccoli. Si chiama virgola **mobile** proprio perché cambiando l'esponente la virgola si sposta.

Anche i **caratteri** sono numeri. Il codice **ASCII** assegna un numero a ogni carattere, con 7 o 8 bit per carattere (cioè 128 o 256 caratteri); deve rappresentare:

- le 26 lettere, maiuscole e minuscole (e quelle accentate);
- le 10 cifre e la punteggiatura;
- i caratteri speciali: `CR` (ritorno a inizio riga), `LF` (riga nuova), `TAB`, `DEL`, …

carattere	**A**	**B**	**a**	**c**	**m**
codice	65	66	97	99	109

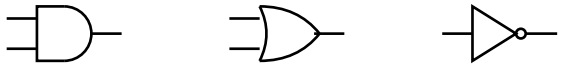
Le lettere hanno codici in ordine alfabetico, quindi confrontare due stringhe (in C `char *s1, *s2`) vuol dire confrontare i codici carattere per carattere: «ciao» viene prima di «mondo» perché `c` = 99 < `m` = 109.

2 Reti logiche

2.1 Porte logiche e tabelle di verità

Una **rete combinatoria** è un circuito che calcola una funzione: l'uscita dipende solo dal valore degli ingressi in quel momento.

I circuiti digitali lavorano su $\{0, 1\}$ con tre operazioni di base, le **porte logiche**. Gli ingressi entrano da sinistra, l'uscita esce a destra.



AND

x	y	z
0	0	0
0	1	0
1	0	0
1	1	**1**

OR

x	y	z
0	0	0
0	1	**1**
1	0	**1**
1	1	**1**

NOT

x	z
0	**1**
1	0

AND vale 1 solo se **tutti** gli ingressi valgono 1; OR vale 1 se **almeno uno** vale 1; NOT inverte. Nelle formule si usa una notazione più corta, come nell'algebra:

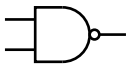
	AND	**OR**	**NOT**
si scrive	$x \cdot y$ oppure xy (prodotto)	$x + y$ (somma)	$\overline{x}$

Somiglia davvero ad aritmetica: su 0 e 1, AND è il prodotto. OR è la somma, tranne $1 + 1$ che fa 1. Alcune proprietà:

associativa	$(x \text{ AND } y) \text{ AND } z \equiv x \text{ AND } (y \text{ AND } z)$	
commutativa	$x \text{ AND } y \equiv y \text{ AND } x$	
elemento neutro	$x \text{ AND } 1 \equiv x$	$x \text{ OR } 0 \equiv x$
elemento assorbente	$x \text{ AND } 0 \equiv 0$	$x \text{ OR } 1 \equiv 1$

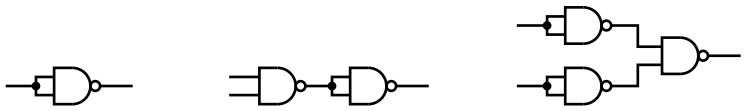
▶ 09 · 24:21 → 29:54

Servono davvero tre tipi di porte? No: ne basta uno solo, come avere un unico mattoncino Lego al posto di tre, che si progetta una volta e poi si collega. Questo mattoncino è la **NAND**: un AND con l'uscita negata (il pallino in fondo, come nel NOT).



x	y	NAND
0	0	**1**
0	1	**1**
1	0	**1**
1	1	0

- **NOT**: $\text{NAND}(x, x)$, perché $x \text{ AND } x = x$ e poi viene negato.
- **AND**: una NAND seguita da un NOT, che toglie la negazione.
- **OR**: per la legge di **De Morgan** $x + y = \overline{\overline{x} \cdot \overline{y}}$, cioè nego i due ingressi e li mando in una NAND.



NOT: 1 NAND AND: 2 NAND OR: 3 NAND

Il prezzo: si risparmia lavoro di progettazione, ma non spazio sul chip, perché per ogni AND o OR servono più NAND.

Per costruire una rete da zero si seguono cinque passi, il **procedimento standard**:

1. descrizione a parole → 2. tabella di verità → 3. **somma di prodotti** OR di tanti AND → 4. rete di porte → 5. tempo di stabilizzazione

La tabella di verità è la funzione descritta caso per caso: per ogni combinazione di ingressi dice quanto vale l'uscita.

Esempio: il **MUX**. A parole: *scegli fra due ingressi x e y a seconda di un ingresso di controllo c.*

```
if (c == 0) z = x;
else        z = y;
```

Dalla descrizione si ricava la tabella di verità. Poi, per ogni riga dove $z = 1$, scrivo un prodotto (AND) che vale 1 **solo** in quella riga: la variabile così com'è se vale 1, negata ($\overline{x}$) se vale 0.

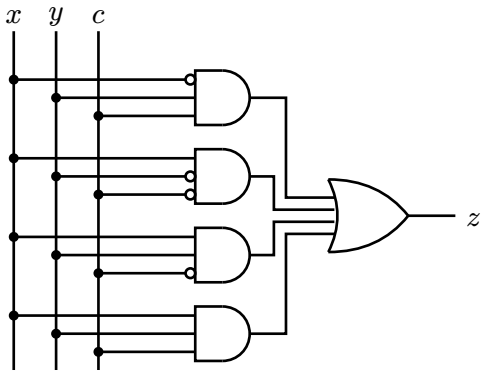
x	y	c	z	**prodotto**
0	0	0	0	
0	0	1	0	
0	1	0	0	
0	1	1	**1**	$\overline{x}\,y\,c$
1	0	0	**1**	$x\,\overline{y}\,\overline{c}$
1	0	1	0	
1	1	0	**1**	$x\,y\,\overline{c}$
1	1	1	**1**	$x\,y\,c$

Le righe con $z = 1$ sono unite da un OR. Ogni prodotto vale 1 solo nella sua riga, quindi l'OR vale 1 esattamente nelle righe scelte.

$$z = \overline{x}\,y\,c + x\,\overline{y}\,\overline{c} + x\,y\,\overline{c} + x\,y\,c$$

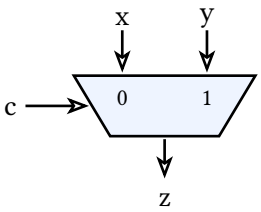
Per esempio $\overline{x}yc$ = NOT(x) AND y AND c: vale 1 solo per $x = 0, y = 1, c = 1$.

Ogni prodotto diventa una porta AND a tre ingressi (il pallino sull'ingresso è un NOT), e le quattro uscite entrano in una OR:



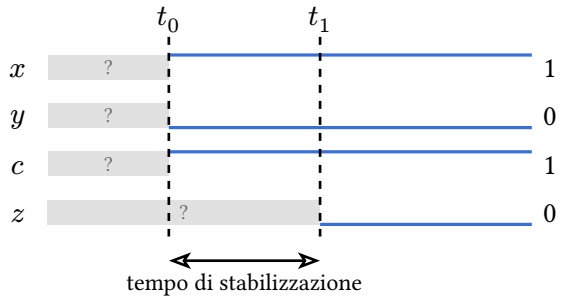
Il MUX come rete di porte

Nei disegni il MUX si abbrevia con un trapezio: in alto i due ingressi, ognuno col valore di c che lo sceglie; a sinistra il controllo; in basso l'uscita.



2.2 Tempo di stabilizzazione

▶ 07 · 01:15 → 06:41 Una porta non risponde subito. Da quando gli ingressi hanno il loro valore (istante t_0) a quando l'uscita è quella giusta (t_1) passa del tempo, e nel frattempo l'uscita può valere qualunque cosa. È il **tempo di stabilizzazione** (o ritardo) della rete.



Il MUX con $c = 1$ fa passare $y = 0$. In grigio i tratti dove il valore non è ancora certo: gli ingressi sono pronti da t_0, ma z è affidabile solo da t_1 in poi

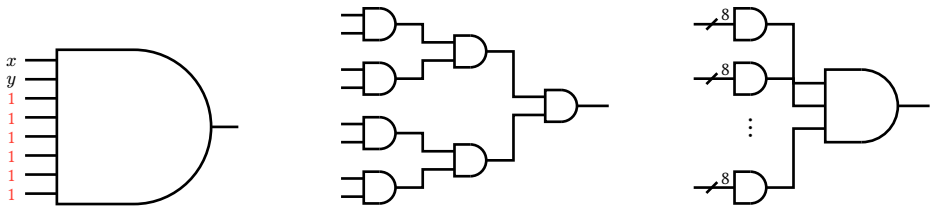
▶ 07 · 06:41 → 17:28 Il ritardo di una porta dipende da quanti ingressi ha: resta quasi uguale fino a una certa soglia, poi cresce in fretta (grafico a sinistra). Nel corso si usa questa convenzione: fino a 8 ingressi, una porta AND o OR ha sempre lo stesso ritardo, che si chiama Δt («delta t»: la lettera greca Δ indica un intervallo, qui di tempo). I tempi delle reti si contano in Δt: $2\Delta t$ vuol dire due porte una dopo l'altra.



- AND e OR **fino a 8 ingressi**: ritardo Δt
- NOT: ritardo **0**

Il NOT non costa perché il segnale negato è già pronto dentro la porta che lo produce: basta prenderlo da un altro punto del circuito.

Con le porte da 8 si fanno anche quelle più piccole: gli ingressi che avanzano si collegano al valore che non cambia il risultato, 1 per l'AND e 0 per l'OR. Per più di 8 ingressi si collegano più porte **ad albero**, e ogni livello dell'albero costa Δt.



AND da 2 con una porta da 8: sei ingressi fissi a 1

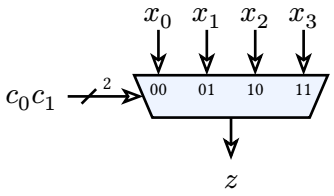
AND da 8 con porte da 2: $\log_2 8 = 3$ livelli

AND da 64 con porte da 8: $\log_8 64 = 2$ livelli

In generale, una porta da n ingressi fatta con porte da k ingressi richiede $\lceil \log_k n \rceil$ livelli ($\lceil \cdot \rceil$ = arrotondato per eccesso): la base è il numero di ingressi di una porta, l'argomento il numero di ingressi totali.

Il tempo di stabilizzazione di una rete si conta sui **livelli** di AND e OR che il segnale attraversa: le porte dello stesso livello lavorano insieme e costano un solo Δt, i livelli uno dopo l'altro si sommano. Il MUX ha un livello di AND e uno di OR: $2\Delta t$. Vale per ogni somma di prodotti, finché termini e variabili non sono più di 8.

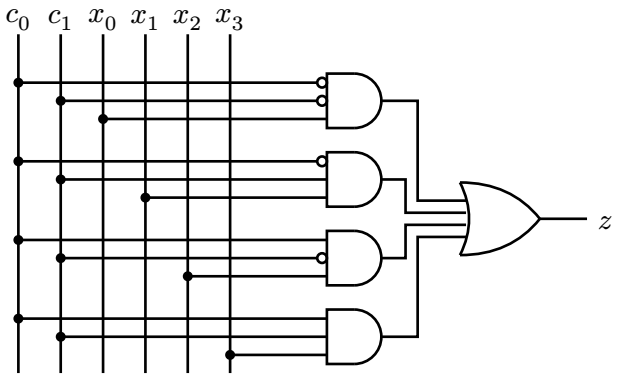
▶ 07 · 17:48 → 37:59 Una rete si può costruire in due modi: da zero con i cinque passi, oppure **componendo** reti già fatte. Esempio: un MUX che sceglie fra **quattro** ingressi con due bit di controllo c_0c_1 (qui c_0 è quello più a sinistra). Il simbolo è quello del MUX a due, con quattro ingressi: sopra ognuno c'è il valore di c_0c_1 che lo sceglie (00 sceglie x_0, 11 sceglie x_3). La barretta col 2 vuol dire che su quel filo passano 2 bit, c_0 e c_1.



c_0	c_1	x_0	x_1	x_2	x_3	z
0	0	0	–	–	–	0
0	0	1	–	–	–	**1**
0	1	–	0	–	–	0
0	1	–	1	–	–	**1**
1	0	–	–	0	–	0
1	0	–	–	1	–	**1**
1	1	–	–	–	0	0
1	1	–	–	–	1	**1**

Da zero. Gli ingressi sono 6: la tabella avrebbe $2^6 = 64$ righe. Si accorcia scrivendo – dove il valore di un ingresso non conta: con controllo 00 conta solo x_0. Ogni riga con tre – ne riassume $2^3 = 8$.

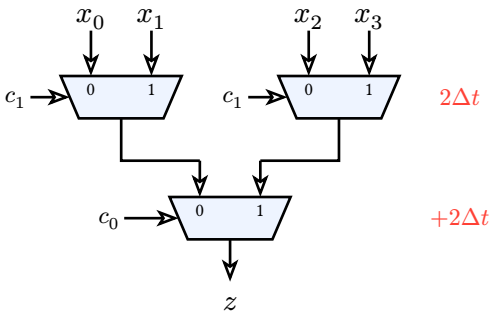
$$z = \overline{c_0}\,\overline{c_1}x_0 + \overline{c_0}c_1x_1 + c_0\overline{c_1}x_2 + c_0c_1x_3$$



Da zero: quattro AND e una OR

Come per il MUX a due, ogni prodotto diventa un AND (qui a sei ingressi, con il pallino sulle variabili negate) e un OR unisce i quattro AND. Il segnale attraversa un livello di AND e uno di OR: $2\Delta t$.

▶ 07 · 38:09 → 49:14 **Componendo.** Scegliere fra quattro vuol dire scegliere prima dentro ogni coppia, poi fra le due coppie. Bastano tre MUX a due ingressi: il bit meno significativo c_1 sceglie dentro le coppie (x_0 o x_1, x_2 o x_3), il più significativo c_0 sceglie quale coppia esce. Non serve la tabella, ma il segnale attraversa due MUX di fila: i due in alto lavorano insieme ($2\Delta t$), quello in basso deve aspettarli ($+2\Delta t$).



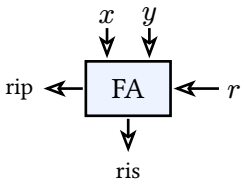
Componendo tre MUX: $4\Delta t$

> **Regola generale**: la rete progettata da zero ha un tempo di stabilizzazione minore o uguale a quella ottenuta componendo, perché è ottimizzata tutta insieme. Comporre però costa molta meno fatica.

2.3 Componenti

Il MUX a quattro è venuto fuori collegando MUX a due già pronti, senza ridisegnarne le porte. È l'idea dei **componenti**: le reti che servono spesso si progettano una volta sola, si disegnano come un blocco col loro nome (come il trapezio del MUX) e poi si collegano fra loro. Oltre al MUX ne servono altri quattro: sommatore, demultiplexer, comparatore e codificatore.

▶ 08 · 00:52 → 13:44 Il **sommatore** fa le somme binarie viste nel capitolo 1. Per costruirlo si guarda la somma in colonna (esempio qui sotto): ogni colonna fa la stessa cosa, cioè somma i due bit x e y e il riporto r arrivato da destra, scrive un bit del risultato e passa un riporto alla colonna a sinistra. Basta quindi una rete che fa una colonna, il **full adder** (FA), ripetuta per ogni colonna.



x	y	r	ris	rip
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

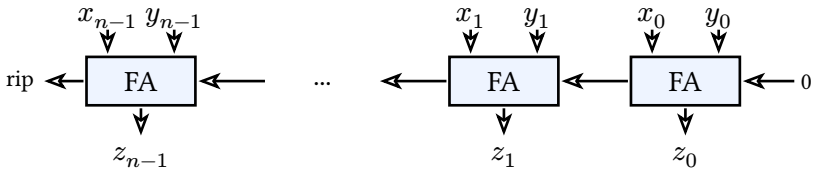
```
 0100
 0011
+0010
 0101
```

3 + 2: ogni colonna somma due bit e il riporto (in grigio)

$$\text{ris} = \overline{x}\,\overline{y}\,r + \overline{x}\,y\,\overline{r} + x\,\overline{y}\,\overline{r} + x\,y\,r$$
$$\text{rip} = \overline{x}\,y\,r + x\,\overline{y}\,r + x\,y\,\overline{r} + x\,y\,r$$

Ogni formula è un OR di quattro prodotti, e ogni prodotto è un AND di tre variabili. Nessuna porta supera gli 8 ingressi (gli AND ne hanno 3, l'OR 4), quindi ognuna costa Δt: il segnale passa per un AND e poi per l'OR, e il FA si stabilizza in $2\Delta t$ (i NOT non costano).

▶ 08 · 14:41 → 17:25 Per sommare numeri di n bit si mettono n FA **in cascata**: il riporto di ognuno entra nel successivo. Da qui in poi i bit si numerano col loro peso: x_0 è il meno significativo, x_{n-1} il più significativo.



Sommatore a n bit: ogni FA aspetta il riporto di quello alla sua destra

Ogni FA deve aspettare il riporto del precedente, quindi i ritardi **si sommano**: $n \cdot 2\Delta t = 2n\Delta t$. Con 2 bit sono $4\Delta t$. Se l'ultimo riporto vale 1, la somma di due numeri senza segno non sta in n bit.

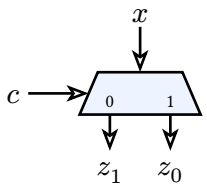
▶ 08 · 19:01 → 24:10 Il sommatore fatto con i FA costa $2n\Delta t$. Come per il MUX a quattro, ci si può chiedere se progettarlo **da zero**, con una sola tabella di verità, sarebbe più veloce. Gli ingressi della tabella sono tutti i bit di x e di y, e la rete è una somma di prodotti: un livello di AND, poi l'OR. Per il tempo basta sapere quanti ingressi hanno queste porte:

- **AND**: ogni prodotto contiene tutte le variabili, quindi ha tanti ingressi quanti la tabella;
- **OR**: riceve un prodotto per ogni 1 della colonna d'uscita. Gli 1 sono al massimo le righe meno una (se fossero tutti 1 l'uscita varrebbe sempre 1, e la rete non servirebbe).

Con 2 bit, contando anche il riporto iniziale, gli ingressi sono 5 e le righe 32. Gli AND hanno 5 ingressi: 1 livello. L'OR ne ha fino a 31, più di 8, quindi va fatto ad albero: 2 livelli. In tutto $3\Delta t$, contro i $4\Delta t$ dei due FA in cascata.

> Se in una colonna gli 1 sono più della metà, conviene scrivere i termini per gli **0** e negare l'uscita, tanto il NOT non costa: i termini sono al massimo la metà delle righe.

▶ 08 · 24:41 → 29:35 Il secondo componente, il **demultiplexer**, è il contrario del MUX: un ingresso x e più uscite. Il controllo sceglie su quale uscita mandare x; le altre valgono 0. Servirà per scegliere in quale cella di memoria scrivere. Si disegna come un MUX capovolto: sotto ogni uscita c'è il valore di c che la sceglie.

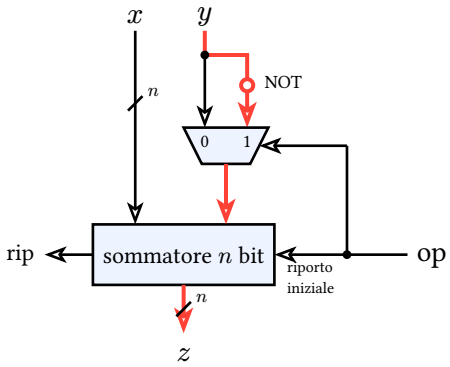


c	x	z_1	z_0
0	0	0	0
0	1	**1**	0
1	0	0	0
1	1	0	**1**

$$z_1 = \overline{c}x \qquad z_0 = cx$$

Ogni uscita ha un solo 1, quindi un solo termine: l'OR non serve. C'è solo il livello di AND: $1\Delta t$.

▶ 08 · 29:42 → 35:18 Lo stesso sommatore fa anche la sottrazione: $x - y = x + \overline{y} + 1$, perché in complemento a 2 il negativo di y è $\overline{y} + 1$. Il bit di controllo **op** fa due cose insieme: comanda il MUX, che con op = 1 fa passare $\overline{y}$, ed entra nel sommatore come **riporto iniziale**, cioè il riporto del primo FA. Con op = 1 quel riporto vale 1, ed è il $+1$.



op = 0 somma, op = 1 sottrazione.
In rosso il cammino critico

Il tempo di stabilizzazione si legge sul **cammino critico** (*critical path*), il percorso più lento dagli ingressi alle uscite:

pezzo	**ritardo**
NOT	0
MUX a n bit	$2\Delta t$
sommatore a n bit	$2n\Delta t$
totale	$\mathbf{2(n+1)\Delta t}$

Il MUX a n bit è fatto di n MUX a un bit con lo stesso controllo: lavorano tutti insieme, quindi costa $2\Delta t$ come uno solo.

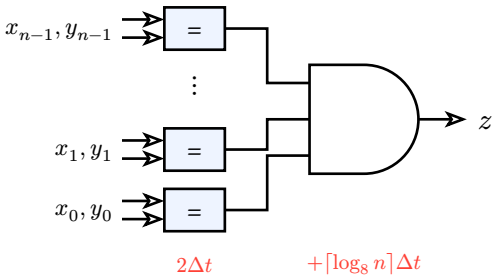
▶ 09 · 04:50 → 15:48 Si torna al sommatore da zero, ora con n bit. La tabella non si scrive più, ma i livelli si contano allo stesso modo, con $\lceil \log_8 \text{ingressi} \rceil$ livelli per ogni porta:

	2 bit	**8 bit**	**n bit**
ingressi	5 (con il riporto iniziale)	16	$2n$
righe	$2^5 = 32$	$2^{16} = 65536$	2^{2n}
livelli di AND	1 (5 ingressi)	$\lceil \log_8 16 \rceil = 2$	$\lceil \log_8 2n \rceil$
livelli di OR	2 (fino a 31 ingressi)	$\lceil \frac{16}{3} \rceil = 6$	$\lceil \log_8 2^{2n} \rceil = \lceil \frac{2n}{3} \rceil$
da zero	$\mathbf{3\Delta t}$	$\mathbf{8\Delta t}$	
con i FA	$4\Delta t$	$16\Delta t$	$2n\Delta t$

Per l'ultima colonna: $\log_8 a = \frac{\log_2 a}{\log_2 8} = \frac{\log_2 a}{3}$. Da zero si va più veloci, ma già con 8 bit la tabella ha 65536 righe: nessuno la scrive, e si usano i FA.

▶ 09 · 16:08 → 19:59 Il terzo componente, il **comparatore**, dice se due numeri di n bit sono uguali: uscita 1 se sono uguali, 0 altrimenti. Per un bit la formula è $z = \overline{x}\,\overline{y} + xy$, che non si semplifica: $2\Delta t$. È il negato dello **XOR** (OR esclusivo), che vale 1 quando i due bit sono diversi. Per n bit si confronta ogni coppia x_i, y_i (nel disegno, ogni riquadro = è il comparatore di un bit) e si fa l'AND di tutte le risposte: basta una coppia diversa per dare 0. Avendo una porta già pronta per il confronto di un bit, con ritardo Δt, si guadagnerebbe solo $1\Delta t$: il livello dell'AND finale resta.

x	y	z
0	0	**1**
0	1	0
1	0	0
1	1	**1**



Comparatore a n bit: $(2 + \lceil \log_8 n \rceil)\Delta t$

▶ 09 · 20:06 → 24:14 L'ultimo componente, il **codificatore**, ha n ingressi di cui **esattamente uno** vale 1, e $\log_2 n$ uscite che dicono in che posizione sta quell'1 (di solito 2^k ingressi e k uscite). Le altre combinazioni di ingressi non sono ammesse, quindi non compaiono nella tabella.

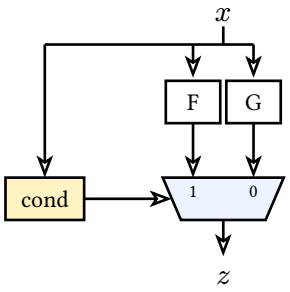
x_3	x_2	x_1	x_0	z_1	z_0
0	0	0	1	0	0
0	0	1	0	0	1
0	1	0	0	1	0
1	0	0	0	1	1

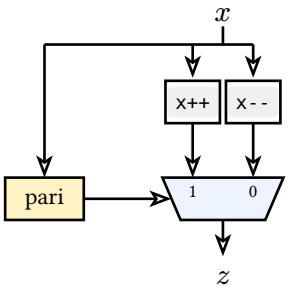
$$z_1 = \overline{x_3} x_2 \overline{x_1}\,\overline{x_0} + x_3 \overline{x_2}\,\overline{x_1}\,\overline{x_0}$$
$$z_0 = \overline{x_3}\,\overline{x_2} x_1 \overline{x_0} + x_3 \overline{x_2}\,\overline{x_1}\,\overline{x_0}$$

L'ultimo termine è uguale nelle due formule: si calcola una volta sola, 3 AND invece di 4. Il tempo resta $2\Delta t$, ma una porta in meno vuol dire meno spazio e meno consumo.

> All'esame $2\Delta t$ è la risposta giusta solo se i livelli sono due. Conta sempre i livelli: il demultiplexer ne ha uno ($1\Delta t$), due FA in cascata ne hanno quattro ($4\Delta t$).

▶ 09 · 31:05 → 40:43 Costruire una rete collegando componenti, come il MUX a quattro o il circuito che somma e sottrae, si chiama **approccio strutturale**. Si parte da uno pseudocodice fatto di funzioni e scelte, e ogni pezzo diventa un componente: ogni funzione è una rete, la condizione è una rete con un bit d'uscita, e l'`if` è un MUX comandato da quel bit. Nei disegni x entra in tutti i blocchi; il MUX fa uscire il risultato di F se la condizione vale 1 (qui l'ingresso 1 è a sinistra), quello di G se vale 0.





`if (cond) f(x) else g(x)` `if (pari(x)) x++ else x--`

Nell'esempio di destra, con x di n bit, i tre blocchi si fanno con quello che c'è già:

- `x++`: un sommatore con ingressi x e la costante 1, riporto iniziale 0. Una costante è un filo fisso a 0 o a 1;
- `x--`: un sommatore con x e la costante 1 negata, riporto iniziale 1: è $x - 1$ fatto come sottrazione;
- `pari(x)`: un numero è pari quando il suo ultimo bit x_0 vale 0, quindi basta un NOT su x_0, senza calcoli.

2.4 Semplificare le formule

▶ 10 · 00:29 → 07:10 Fra la somma di prodotti e la rete si può **semplificare** la formula con le regole dell'algebra. Si raccoglie, poi si usano $t + \overline{t} = 1$ e $x \cdot 1 = x$:

$$z = xyt + xy\overline{t} = xy(t + \overline{t}) = xy$$

Qui l'OR sparisce: da $2\Delta t$ a $1\Delta t$. Un termine si può anche usare due volte, perché $x + x = x$:

$$z = xyt + xy\bar{t} + \bar{x}yt = \underbrace{xyt + xy\bar{t}}_{xy} + \underbrace{xyt + \bar{x}yt}_{yt} = xy + yt$$

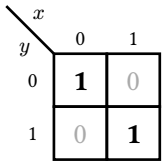
I livelli restano due, quindi il tempo non cambia, ma le porte calano: da 3 AND e 1 OR a 2 AND e 1 OR. Con porte da soli 2 ingressi il guadagno è più grande, perché ogni AND da tre ingressi diventa due porte: da 8 porte a 3. Raccogliendo ancora, $z = y(x + t)$: 2 porte.

Semplificare quindi non sempre fa guadagnare tempo, ma fa risparmiare porte: meno spazio e meno consumo. Allo stesso modo il riporto del FA, usando tre volte il termine xyr, diventa:

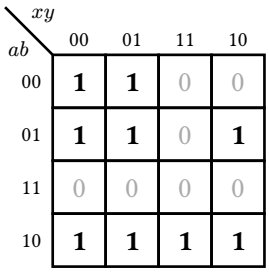
$$\text{rip} = yr + xr + xy$$

▶ 10 · 07:13
→ 36:00

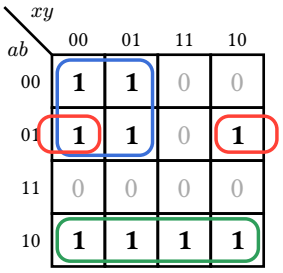
Per vedere cosa raccogliere si usano le **mappe di Karnaugh**: sono la tabella di verità ridisegnata come griglia, con alcune variabili sulle colonne e le altre sulle righe. Le combinazioni sono scritte nell'ordine 00, 01, 11, 10, così fra due celle vicine cambia **un solo bit**. Nell'angolo in alto a sinistra, sopra la diagonale ci sono le variabili delle colonne, sotto quelle delle righe; ogni cella è il valore dell'uscita per quella combinazione (gli 0 in grigio).



2 variabili:
il comparatore a un bit



4 variabili:
una funzione di esempio



gli stessi 1 raccolti in tre gruppi

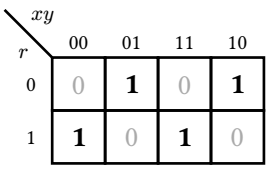
Sulla mappa si cercano **gruppi di 2^k celle vicine, tutte a 1, a forma di quadrato o rettangolo**. Ogni gruppo diventa un solo termine: ci restano le variabili che nel gruppo non cambiano, le altre k spariscono.

- Gruppo blu, 4 celle: y e b cambiano, x e a valgono sempre 0. Termine $\bar{x}\,\bar{a}$.
- Gruppo verde, 4 celle: tutta la riga $ab = 10$, x e y cambiano. Termine $a\bar{b}$.
- Gruppo rosso, 2 celle: la mappa **si richiude sui bordi**, la prima colonna è vicina all'ultima (e la prima riga all'ultima). Cambia solo x. Termine $\bar{y}\,\bar{a}b$.

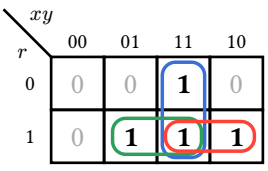
$$z = \bar{x}\,\bar{a} + a\bar{b} + \bar{y}\,\bar{a}b$$

Un 1 isolato resta un termine con tutte le variabili; su 4 variabili un gruppo da 8 ne lascia una sola. Coprendo tutti gli 1 con il minor numero di gruppi, i più grandi possibili, si ottiene la formula più semplice. I gruppi si chiamano **implicanti**.

Le due uscite del full adder:

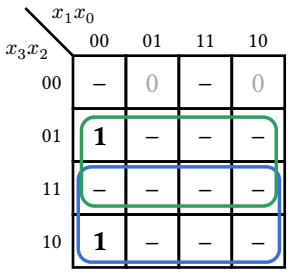


ris: gli 1 sono tutti isolati,
restano i quattro termini da tre variabili

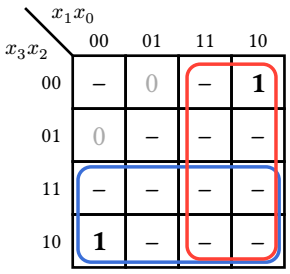


rip: tre gruppi da 2,
rip $= xy + yr + xr$

Nel codificatore molte combinazioni di ingressi non sono ammesse: sulla mappa si segnano con –. Non capiteranno mai, quindi si possono contare come 1 quando fa comodo per ingrandire un gruppo.



$z_1 = x_3 + x_2$



$z_0 = x_3 + x_1$

Due gruppi da 8 per ogni uscita: spariscono tre variabili su quattro, e ogni uscita diventa un solo OR.

Con 5 variabili servirebbero due mappe 4×4 una sopra l'altra, una per ogni valore della quinta variabile. Per questo oltre le 4 variabili le mappe non si usano.

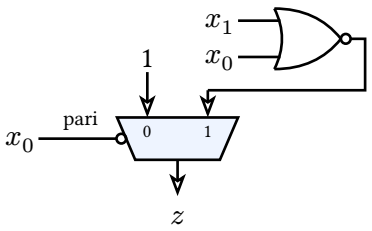
▶ 12 · 05:44 → 19:21

Due esempi mettono insieme tutto. Il primo, su x di 4 bit $x_3x_2x_1x_0$:

```
if (x è pari)  { if (x è divisibile per 4) z = 1; else z = 0; }
else           z = 1;
```

Con l'approccio strutturale servono due condizioni:

- x pari: $x_0 = 0$, cioè $\overline{x_0}$;
- x divisibile per 4: dividere per $4 = 2^2$ vuol dire spostare i bit di due posti a destra (*shift*). I due bit che escono, x_1x_0, sono il resto $x \bmod 4$: deve essere 00. «Tutti e due 0» è un OR negato (NOR): l'OR vale 0 solo se entrambi sono 0. Per De Morgan è lo stesso che $\overline{x_1} \cdot \overline{x_0}$.



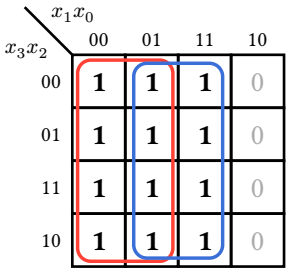
Due `if` ⟶ due MUX; il secondo si riduce a una NOR

Ogni `if` è un MUX. Quello interno sceglie 1 se la NOR vale 1 e 0 se vale 0: la sua uscita è la NOR stessa, quindi il MUX si butta e resta la NOR. Quello esterno è comandato da $\overline{x_0}$: dispari ⟶ 1, pari ⟶ la NOR.

Altra semplificazione: nel ramo «pari» si sa già che $x_0 = 0$, quindi basta controllare $x_1 = 0$.

Con tabella e mappa: $z = 1$ se $x_0 = 1$ (dispari) oppure se $x_1x_0 = 00$. Ogni condizione riassume 8 righe della tabella:

x_3	x_2	x_1	x_0	z
–	–	–	1	1
–	–	0	0	1



Dalla tabella: $z = x_0 + \overline{x_1}\,\overline{x_0}$.
Sulla mappa due gruppi da 8, che **possono sovrapporsi**: basta che coprano tutti gli 1, i più grandi possibile.
$z = \overline{x_1} + x_0$: più semplice.

▶ 12 · 19:31 → 32:14

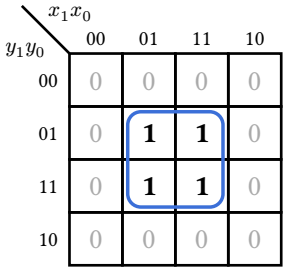
Il secondo esempio è un **moltiplicatore** di due numeri senza segno da 2 bit: $x_1x_0 \cdot y_1y_0 = z_3z_2z_1z_0$. L'uscita ha 4 bit, perché moltiplicando i bit raddoppiano: $3 \cdot 3 = 9 = 1001$.

x_1	x_0	y_1	y_0	z_3	z_2	z_1	z_0
0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	0
0	0	1	0	0	0	0	0
0	0	1	1	0	0	0	0
0	1	0	0	0	0	0	0
0	1	0	1	0	0	0	1
0	1	1	0	0	0	1	0
0	1	1	1	0	0	1	1
1	0	0	0	0	0	0	0
1	0	0	1	0	0	1	0
1	0	1	0	0	1	0	0
1	0	1	1	0	1	1	0
1	1	0	0	0	0	0	0
1	1	0	1	0	0	1	1
1	1	1	0	0	1	1	0
1	1	1	1	1	0	0	1

Se $x = 0$ il prodotto è 0; se $x = 1$ si ricopia y; e così via. Si cercano poi gli 1 in righe che differiscono per **un solo bit**: quel bit si raccoglie e sparisce, come su una mappa.

$$z_3 = x_1 x_0 y_1 y_0$$
$$z_2 = x_1 \overline{x_0} y_1 + x_1 y_1 \overline{y_0}$$
$$z_1 = \overline{x_1} x_0 y_1 + x_1 \overline{x_0} y_0 + x_1 x_0 \overline{y_1} y_0 + x_1 x_0 y_1 \overline{y_0}$$
$$z_0 = \overline{x_1} x_0 y_0 + x_1 x_0 y_0 = x_0 y_0$$

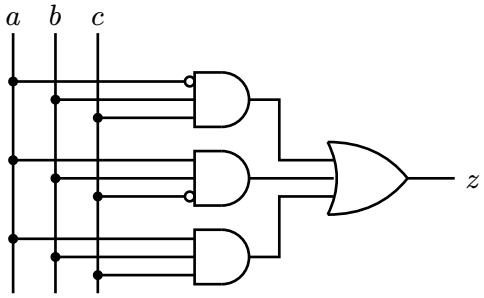
Per z_2: le righe 1010 e 1011 differiscono solo per y_0, e danno $x_1 \overline{x_0} y_1$; 1010 e 1110 solo per x_0, e danno $x_1 y_1 \overline{y_0}$.



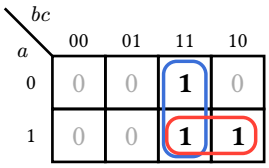
Per z_0 i due termini stanno lontani nella tabella (righe 0101-0111 e 1101-1111), e si vede che si raccolgono solo facendo il conto. Sulla mappa i quattro 1 sono un quadrato: $z_0 = x_0 y_0$ subito. È a questo che servono le mappe.

▶ 15 · 00:33 → 14:33

Si può anche fare il contrario: partire da una rete già disegnata, scriverne la formula e chiedersi se si può calcolare la stessa cosa in modo più efficiente. Questa rete ha un AND per ogni termine (il pallino nega l'ingresso) e una OR che li raccoglie:



La rete da leggere



bc: la colonna 11, a cambia
ab: la riga $a = 1$, c cambia

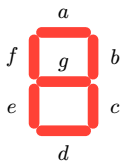
Il terzo termine differisce dal primo solo per a e dal secondo solo per c: lo si usa due volte.

$$z = \overline{a}bc + ab\overline{c} + abc = \underbrace{\overline{a}bc + abc}_{bc} + \underbrace{ab\overline{c} + abc}_{ab} = bc + ab = b(c + a)$$

La mappa dà subito $bc + ab$: 2 AND e una OR. Il raccoglimento $b(c + a)$ invece sulla mappa non si vede: lì si cercano solo i gruppi più grandi. Raccogliendo resta una OR e un AND, una porta in meno. I livelli sono sempre due, quindi $2\Delta t$. La rete di partenza ha due livelli solo con porte da 8 ingressi: con porte da 2, l'AND da tre e l'OR da tre diventano due livelli ciascuno, e i livelli sono 4.

▶ 15 · 14:41 → 29:37

Le mappe convengono soprattutto quando gli 1 sono tanti. Ne è un esempio il **display a sette segmenti** di calcolatrici e orologi: ogni cifra si forma accendendo alcuni dei sette segmenti, che per convenzione si chiamano $a \ldots g$ (a in alto, poi in senso orario, g al centro; il punto decimale qui non serve). La rete riceve una cifra da 0 a 9 in binario, $x_3 x_2 x_1 x_0$, e dà 1 sui segmenti da accendere e 0 su quelli da lasciare spenti.



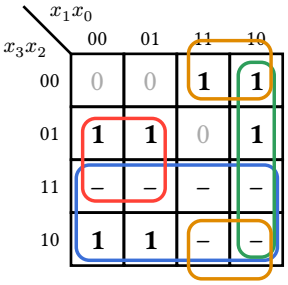
I nomi dei segmenti



Il 4 = 0100: f, b, g, c accesi

cifra	x_3	x_2	x_1	x_0	a	b	c	d	e	f	g
0	0	0	0	0	1	1	1	1	1	1	0
1	0	0	0	1	0	1	1	0	0	0	0
2	0	0	1	0	1	1	0	1	1	0	1
3	0	0	1	1	1	1	1	1	0	0	1
4	0	1	0	0	0	1	1	0	0	1	1
5	0	1	0	1	1	0	1	1	0	1	1
6	0	1	1	0	1	0	1	1	1	1	1
7	0	1	1	1	1	1	1	0	0	0	0
8	1	0	0	0	1	1	1	1	1	1	1
9	1	0	0	1	1	1	1	1	0	1	1

Sette colonne d'uscita sono sette reti combinatorie, ognuna con 4 ingressi, e quindi sette mappe. Per g la somma di prodotti avrebbe 7 termini da 4 variabili ($\overline{x_3}\,\overline{x_2}x_1\overline{x_0} + \ldots$). Sulla mappa conviene mettere prima i tre 0 (cifre 0, 1 e 7) e riempire il resto. Le combinazioni da 1010 a 1111 (da 10 a 15) non sono cifre e non capitano mai: sono – come nel codificatore, e si contano come 1 dove serve.



- blu, 8 celle: le righe 11 e 10, conta solo $x_3 = 1$. Termine x_3.
- verde, 4 celle: la colonna 10. Termine $x_1\overline{x_0}$.
- rosso, 4 celle: righe 01 e 11, colonne 00 e 01. Termine $\overline{x_1}x_2$.
- arancio, 4 celle: righe 00 e 10 (la mappa si richiude sopra e sotto), colonne 11 e 10. Termine $x_1\overline{x_2}$.

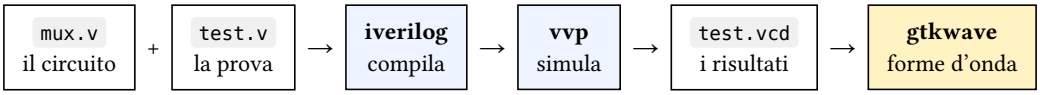
$$g = x_3 + x_1\overline{x_0} + \overline{x_1}x_2 + x_1\overline{x_2}$$

I gruppi si sovrappongono, ma va bene: un 1 coperto due volte non costa niente, mentre un gruppo da 4 toglie due variabili e uno da 2 una sola. Coprire gli stessi 1 con gruppi da 2 e da 1 darebbe termini più lunghi.

2.5 Descrivere le reti in Verilog

▶ 11 · 00:49 → 08:35

Una rete progettata sulla carta si può provare al calcolatore prima di costruirla, perché stamparla sul silicio è lungo e costoso. Si usa un **HDL** (*Hardware Description Language*): un linguaggio di programmazione che invece di un algoritmo descrive come funziona un circuito. I più usati sono VHDL e **Verilog**; nel corso si usa Verilog (versione 2003, non il SystemVerilog del libro), perché ha strumenti gratuiti per ogni sistema operativo:




```
$ iverilog mux.v test.v      # compila: crea a.out
$ vvp a.out                  # simula: scrive test.vcd (basta anche ./a.out)
$ gtkwave test.vcd           # mostra i segnali nel tempo
```

È lo stesso giro del C: `iverilog` fa quello che fa `gcc`, controlla gli errori e crea `a.out`; `vvp` lo esegue, cioè **simula**: fa finta di essere il circuito e registra cosa fa. A `iverilog` si danno **tutti** i file in cui stanno i moduli usati, in qualunque ordine, come a `gcc` più file `.c`.

Ogni pezzo di circuito è un **modulo**, che si scrive come una funzione: i parametri sono i fili d'ingresso (`input`) e d'uscita (`output`), di solito prima le uscite. Un nome solo vuol dire un filo da 1 bit.

```
module nome(output z, input x, input y);   // testa: il nome e i fili
  ...                                      // corpo: come si calcolano le uscite
endmodule
```

Il MUX si può scrivere in tre modi.

▶ 11 · 09:14 → 13:52

1. Come tabella di verità, con una `primitive`. Ogni riga dà i valori degli ingressi nell'ordine dei parametri, poi `:` e l'uscita; `?` è il valore che non conta (il – delle tabelle).

```
primitive mux(output z, input x, input y, input c);
  table
  // x y c : z
     0 ? 0 : 0;
     1 ? 0 : 1;
     ? 0 1 : 0;
     ? 1 1 : 1;
  endtable
endprimitive
```

> Una `primitive` ha **una sola uscita** da 1 bit, e la tabella deve coprire **tutte** le combinazioni, anche quelle con uscita 0. Se mancano, lì l'uscita non vale 0 ma `x` (non definito).

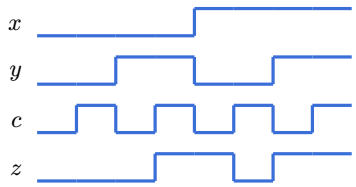
▶ 11 · 14:44 → 23:31

Prima di vedere gli altri due modi, questo MUX si può già provare. Si scrive un **modulo di test**, senza parametri: è il «main». È un banco di prova, e lo schema è sempre lo stesso: un `reg` per ogni ingresso del circuito (un interruttore), un `wire` per ogni uscita (una lampadina), una copia del circuito collegata a quei fili, e il programma che muove gli interruttori.

```
module test;
  reg x, y, c;                 // ingressi
  wire z;                      // uscita
  mux m(z, x, y, c);           // il circuito da provare
  integer i;
  initial begin
    $dumpfile("test.vcd");
    $dumpvars;
    $monitor("x=%b y=%b c=%b z=%b", x, y, c, z);
    for (i = 0; i < 8; i++) begin
      {x, y, c} = i;           // i in binario: x è il bit più a sinistra
      #2;
    end
    $finish;
  end
endmodule
```

riga	cosa fa
`reg x, y, c;`	fili a cui **il test** dà un valore con `=`, e che lo tengono finché non lo cambia: gli ingressi
`wire z;`	un filo e basta: porta quello che gli manda il circuito. È l'uscita, e non gli si può assegnare niente
`mux m(z, x, y, c);`	crea una **copia** (*istanza*) del MUX e la collega ai fili, nell'ordine dei parametri del modulo. Non è una chiamata di funzione: è un pezzo di hardware sempre acceso, che ricalcola `z` ogni volta che cambia un ingresso. `m` è il nome della copia: serve perché se ne possono creare quante se ne vuole
`integer i;`	un intero, come `int` in C
`initial begin … end`	i comandi fra `begin` ed `end` (le graffe del C) si eseguono una volta sola, all'accensione
`$dumpfile("test.vcd");` `$dumpvars;`	registrano nel file ogni cambiamento di ogni segnale: il file si apre con gtkwave, dove i segnali stanno sotto `test` → `m`
`$monitor("…", x, z);`	stampa una riga ogni volta che cambia un segnale della lista. Come nella `printf`, `%b` scrive il valore in binario e `%d` in decimale. `$display` stampa una volta sola
`{x, y, c} = i;`	le graffe **mettono insieme** tre fili in un numero da 3 bit: con `i` da 0 a 7 si provano tutte le combinazioni
`#2;`	aspetta 2 unità di tempo (`#1` vale circa un Δt): il circuito ha il tempo di rispondere
`$finish;`	ferma la simulazione, che altrimenti non finirebbe mai

Con un `if` il test può anche controllare da solo se l'uscita è quella giusta.



Quello che mostra gtkwave per il test: i da 0 a 7. Con $c = 0$ l'uscita copia x, con $c = 1$ copia y

▶ 11 · 25:03 → 30:05

Gli altri due modi di scrivere il MUX hanno gli stessi parametri, quindi il test di prima li prova senza cambiare niente.

2. Come formula, con un `module` e `assign`: ogni volta che cambia un ingresso, ricalcola l'uscita. Un'uscita = un `assign`. È così che si dà un valore a un `wire`, e le uscite di un modulo lo sono; il `=` dentro `initial` vale solo per i `reg`. L'AND lega più dell'OR, come il prodotto sulla somma, quindi qui le parentesi non servono.

```
module mux(output z, input x, input y, input c);
  assign z = !c && x || c && y;
endmodule
```

	AND	OR	NOT
in Verilog	`&&` oppure `&`	`||` oppure `|`	`!` oppure `~`

Le due forme coincidono solo sui fili da 1 bit. Su più bit `&`, `|`, `~` lavorano bit per bit (`~` di 0101 è 1010), mentre `&&`, `||`, `!` trattano il numero come vero (diverso da 0) o falso, come in C, e danno un bit solo (`!` di 0101 è 0).

Si possono usare anche le **porte già pronte** `and`, `or`, `not`: il primo parametro è l'uscita. I collegamenti interni sono fili, dichiarati con `wire`.

```
module mux(output z, input x, input y, input c);
  wire nc, a1, a2;
  not(nc, c);
  and(a1, nc, x);
  and(a2, c, y);
  or(z, a1, a2);
endmodule
```

3. Come in C, con l'espressione `cond ? a : b`: vale `a` se `cond` è vera, altrimenti `b`. Qui gli ingressi hanno **N bit**: `[N-1:0]` vuol dire i bit da $N-1$ a 0. Un bit solo si prende con le quadre: `x[0]` è il meno significativo, `x[N-1]` il più significativo, e `x[3:0]` sono i 4 bit più a destra. Il parametro N ha un valore che si può cambiare per ogni copia del modulo. Nel test, per esempio, `mux #(1) m(z, x, y, c);` crea una copia con N = 1.

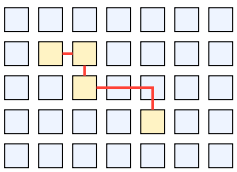
```
module mux #(parameter N = 32)
           (output [N-1:0] z, input [N-1:0] x, input [N-1:0] y, input c);
  assign z = (c == 0 ? x : y);
endmodule
```

> Il prof scrive `parameter N = 32;` dentro il modulo, dopo la testa. Con Icarus 13 così non compila (*Unable to bind parameter*): N è usato nella testa prima di essere dichiarato. Va messo nella testa, con `#(parameter N = 32)` subito dopo il nome.

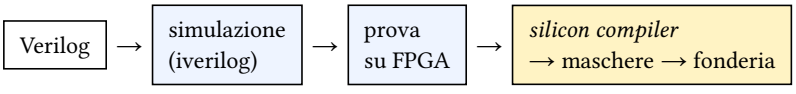
Le costanti si scrivono con bit, base e valore: `1'b0` è 0 su 1 bit in binario, `16'd5` è 5 su 16 bit scritto in decimale.

▶ 11 · 38:19 → 48:45

Dopo la simulazione ci sono altri due passi. Il primo è provare il circuito su una **FPGA**: un chip con una griglia di milioni di celle, dove ogni cella si può configurare come una piccola funzione (4-5 bit d'ingresso, 1 d'uscita), come registro da 1 bit, o come incrocio che instrada i segnali fra le celle vicine. Il Verilog compilato dice come configurare le celle; la configurazione si scrive sul chip come su una chiavetta, e resta anche a corrente spenta. Collegando interruttori ai piedini si prova il circuito vero. L'FPGA si può riprogrammare: oggi un MUX, domani un full adder.



FPGA: in giallo le celle usate, in rosso i segnali fra loro



Il percorso dal Verilog al chip

Quando tutto funziona, un **silicon compiler** prende lo stesso Verilog e produce tutto quello che serve per stampare il chip: le **maschere** da mandare alla fonderia.

▶ 11 · 48:50 → 52:39

I moduli si **compongono**: un modulo può contenere copie di altri moduli. Il full adder ha due uscite, quindi non sta in una `primitive`: si scrivono due tabelle, una per il risultato e una per il riporto, e un modulo che le usa tutte e due sugli stessi ingressi.

```
primitive far(output z, input x, input y, input r);   // risultato
  table  0 0 0 : 0;  0 0 1 : 1;  0 1 0 : 1;  0 1 1 : 0;
         1 0 0 : 1;  1 0 1 : 0;  1 1 0 : 0;  1 1 1 : 1;  endtable
endprimitive

primitive fac(output c, input x, input y, input r);   // riporto
  table  0 0 0 : 0;  0 0 1 : 0;  0 1 0 : 0;  0 1 1 : 1;
         1 0 0 : 0;  1 0 1 : 1;  1 1 0 : 1;  1 1 1 : 1;  endtable
endprimitive

module fa(output c, output z, input x, input y, input r);
  far ris(z, x, y, r);
  fac car(c, x, y, r);
endmodule
```

Ogni copia si collega nell'ordine dei parametri della sua testa: `ris` riceve `z, x, y, r`. Allo stesso modo due `fa` fanno il sommatore a 2 bit, uno per colonna. Il riporto della colonna di destra va a quella di sinistra: è un collegamento interno, quindi un `wire`, su cui una copia scrive e l'altra legge.

```
module add2(output c, output [1:0] z, input r, input [1:0] x, input [1:0] y);
  wire rr;                          // il riporto fra le due copie
  fa fa0(rr, z[0], x[0], y[0], r);  // colonna di destra: bit 0
  fa fa1(c, z[1], x[1], y[1], rr);  // colonna di sinistra: bit 1
endmodule
```

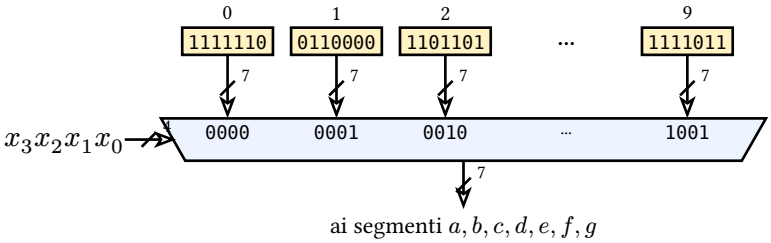
E così via fino a un processore intero, un pezzo alla volta, provando ogni pezzo da solo.

Gli errori più comuni. `iverilog` dice file e riga (`es2.v:4: syntax error`). Si guarda prima l'errore più in alto: spesso gli altri sono solo conseguenze (un `endmodule` scritto male fa sembrare «già dichiarati» tutti i fili del modulo che segue).

- `Unknown module type: fa`: nel comando manca il file dove sta `fa`.
- `'x' is not a valid l-value … declared here as a wire`: si sta assegnando un valore a un `wire`; gli ingressi del test vanno dichiarati `reg`.
- `expects 1 bit(s), given 32` (avviso): uno `0` scritto da solo è un numero da 32 bit; per un filo da 1 bit si scrive `1'b0`.
- Un'uscita che vale `z` è un filo non collegato a niente, per esempio un'uscita del modulo su cui nessuna copia scrive.

▶ 15 · 29:40 → 36:23

Resta da vedere come funziona una cella dell'FPGA. Ogni cella calcola la sua funzione senza porte su misura: tiene la colonna d'uscita della tabella di verità in un registro, e gli ingressi, come controllo di un MUX, scelgono quale bit esce. È un'idea che vale anche fuori dalle FPGA. Per il display a sette segmenti, invece di sette formule, si mettono le dieci righe d'uscita della tabella in dieci registri da 7 bit, collegati agli ingressi di un MUX comandato dalla cifra $x_3x_2x_1x_0$: 4 bit di controllo, quindi 16 ingressi. La cifra 4 = 0100 fa passare il registro con 0110011, e i 7 fili d'uscita vanno ai segmenti. Il ritardo è la lettura del registro più quello del MUX. Gli ingressi da 1010 a 1111 vanno comunque riempiti, per esempio con tutto spento.



La tabella del display in registri: la cifra sceglie quale riga esce

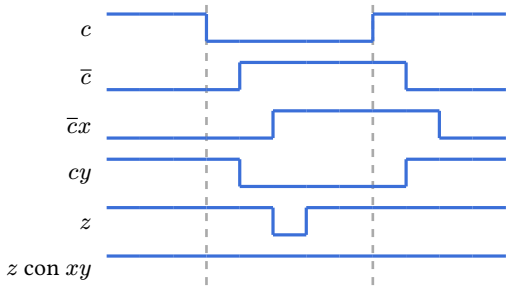
▶ 15 · 38:56 → 52:09

La simulazione mostra anche cose che sulla carta sfuggono. Finora il NOT costava 0, ma in realtà un po' di tempo lo prende. Supponiamo allora che **ogni** porta, NOT compreso, risponda dopo un'unità di tempo. In Verilog si scrive `#1` dopo `assign`: ogni volta che cambia un ingresso, l'uscita si aggiorna un'unità di tempo dopo. Con queste porte «lente» si ricostruisce il MUX:

```
module and_d(output z, input x, input y);
  assign #1 z = x && y;            // or_d e not_d uguali, con || e !
endmodule

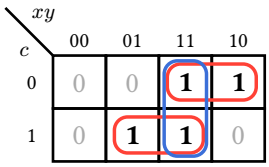
module mux_d(output z, input x, input y, input c);
  wire w1, w2, w3;
  not_d n(w1, c);                  // w1 = not c
  and_d a1(w2, x, w1);             // w2 = not c and x
  and_d a2(w3, y, c);              // w3 = c and y
  or_d  o(z, w2, w3);
endmodule
```

Il test tiene $x = y = 1$ e cambia solo c: l'uscita dovrebbe restare 1, qualunque ingresso si scelga. Ecco invece cosa mostra gtkwave (ogni colonna è un'unità di tempo):



Quando c scende, cy va a 0 dopo un'unità, $\overline{c}x$ sale solo dopo due (NOT e poi AND): per un'unità l'OR vede due 0, e z ha un buco. Quando c risale non succede

Questo disturbo si chiama **alea**: per un attimo l'uscita vale il contrario di quello che dovrebbe. Sulla mappa si vede da dove viene. Con $x = y = 1$ il passaggio è fra due 1 vicini che stanno in due gruppi diversi, $\overline{c}x$ e cy: per un attimo nessuno dei due termini vale 1. Ogni salto di questo tipo può dare un'alea, ma non è detto che succeda.



Si toglie aggiungendo un gruppo che copre i due 1 del passaggio, la colonna $xy = 11$:

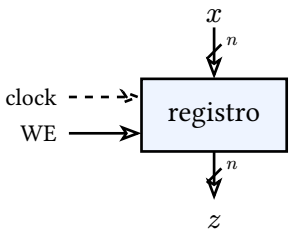
$$z = \overline{c}x + cy + xy$$

Il termine xy non copre nessun 1 nuovo, ma con $x = y = 1$ tiene l'uscita a 1 qualunque cosa faccia c: nell'ultima riga delle forme d'onda il buco non c'è più.

Il prezzo è un AND in più e un ingresso in più nell'OR: con porte da 8 i livelli restano due, con porte da 2 l'OR da tre diventa un albero e il ritardo cresce.

2.6 Memoria: latch, flip-flop, registro

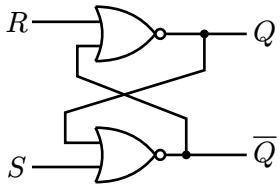
▶ 13 · 06:16 → 11:11 Finora le reti calcolavano funzioni: stesso ingresso, stessa uscita. Una rete **sequenziale** invece **ricorda**: l'uscita dipende dalla sequenza di ingressi ricevuti. Il mattone è un circuito a cui si può dire «ricordati 1» o «ricordati 0», con due uscite: Q, il valore ricordato, e $\overline{Q}$, il suo negato. L'obiettivo è il **registro**:



Se **clock = 1 e WE = 1** (*write enable*, «scrittura abilitata»), il registro si ricorda l'ingresso x di n bit: da lì in poi z vale x. Altrimenti continua a ricordare il valore di prima, come un foglio dove resta scritto l'ultimo numero finché non lo si cancella. Il clock è il segnale che sale e scende a ritmo fisso, visto all'inizio; nei disegni il suo filo è tratteggiato.

Ci si arriva in quattro passi, dal più semplice, facendo prima il circuito per **un bit**.

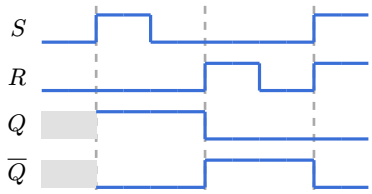
▶ 13 · 12:25 → 27:21 Il primo è il **latch SR** (*Set Reset*): due porte NOR (OR seguito da NOT) con le uscite **incrociate**, cioè l'uscita di ognuna torna indietro come ingresso dell'altra. $S = 1$ vuol dire «ricordati 1», $R = 1$ «ricordati 0».



Latch SR

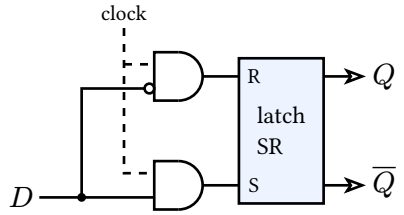
S	R	Q	
0	0	quello di prima	ricorda
1	0	1	*set*
0	1	0	*reset*
1	1	$Q = \overline{Q} = 0$	vietato

Per capire perché funziona si segue il segnale. Con $R = 1$, $S = 0$: la NOR in alto ha un ingresso a 1, quindi dà $Q = 0$. Quello 0 torna nella NOR in basso, che ora ha tutti e due gli ingressi a 0 e dà $\overline{Q} = 1$. A sua volta l'1 torna in alto, dove non cambia niente: il circuito si è **stabilizzato**. Se ora anche R torna a 0, gli ingressi che rientrano dall'altra porta sono ancora quelli di prima, e $Q = 0$ resta. Con $S = 1$ succede lo stesso al contrario, perché il circuito è simmetrico. Il valore resta finché c'è corrente.

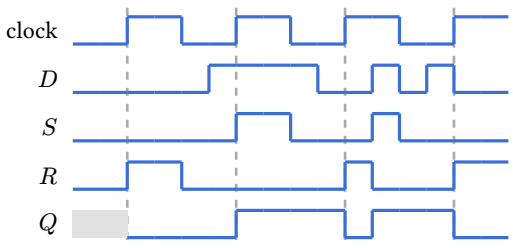


All'inizio Q non è definito (grigio). Set, poi tutto a 0: ricorda 1. Reset: ricorda 0. Alla fine $S = R = 1$: Q e $\overline{Q}$ valgono tutti e due 0

▶ 13 · 29:05 → 40:55 Il problema è proprio $S = R = 1$: entrambe le NOR danno 0, e Q e $\overline{Q}$ non sono più uno il negato dell'altro. Si risolve col secondo passo, il **latch D**: davanti al latch SR si mettono due AND, e due ingressi nuovi, il **clock**, che dice **quando** scrivere (quando vale 1), e D, che dice **cosa** scrivere. A S arriva clock AND D, a R arriva clock AND $\overline{D}$: S e R non possono più valere 1 insieme.



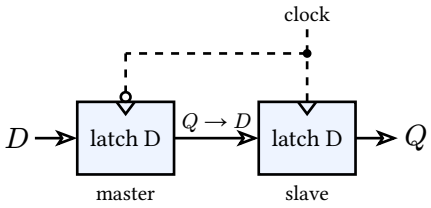
Latch D (il clock si disegna tratteggiato)



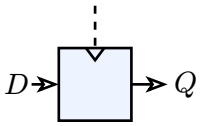
Col clock alto: $D = 0$ fa un reset, $D = 1$ un set. Col clock basso Q non cambia

Ma anche qui c'è un problema: **mentre il clock è 1** il latch copia D in continuazione (si dice che è **trasparente**). Se D cambia mentre il clock è alto, nello stesso ciclo arrivano un set e un reset, e Q cambia due volte (terzo ciclo: prima un reset, poi un set). Invece in un ciclo di clock si deve scrivere un solo valore. Quindi ***D* non può cambiare mentre il clock vale 1**.

▶ 13 · 40:56 → 49:06 Il terzo passo toglie anche questo vincolo: il **flip-flop**, fatto di due latch D in fila. Il Q del primo, il **master**, è il D del secondo, lo **slave**. Lo slave riceve il clock, il master il clock **negato**: quando scrive uno, l'altro è **opaco** (non fa passare niente).

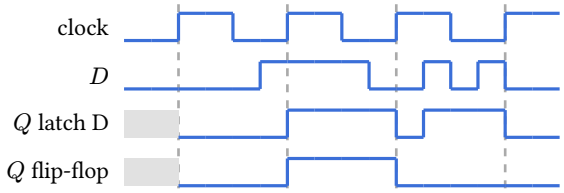


Flip-flop master-slave: il pallino nega il clock del master



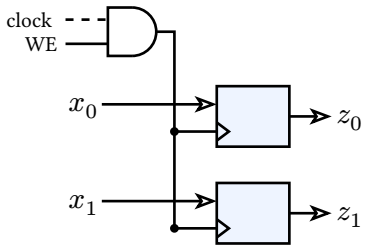
Simbolo: il triangolo è l'ingresso del clock

Col clock basso scrive il master, e lo slave tiene fermo Q. Quando il clock **sale** il master diventa opaco e lo slave copia quello che il master aveva scritto. Non c'è mai un momento in cui scrivono tutti e due, quindi il cammino da D a Q è sempre interrotto da qualche parte: Q cambia **solo sul fronte di salita** del clock, e i cambiamenti di D negli altri momenti non contano.



Stessi ingressi: il latch D segue D per tutto il tempo in cui il clock è alto, il flip-flop prende D solo sul fronte di salita (linee tratteggiate)

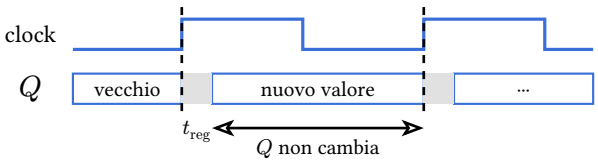
▶ 13 · 49:11 → 51:06 L'ultimo passo è il **write enable**: invece di dare il clock direttamente al flip-flop, gli si dà clock AND WE. Con WE = 0 l'AND dà sempre 0, il flip-flop non vede mai salire il clock e tiene il suo valore. Un **registro da *n* bit** è una fila di n flip-flop, ognuno col suo bit, con lo stesso clock e lo stesso WE.



Registro da 2 bit

Ogni bit costa sempre più porte: il latch SR 2, il latch D 4 (il NOT non si conta), il flip-flop 8, con il WE 9.

Il registro si scrive sul fronte di salita del clock, ma l'uscita è stabile solo dopo un piccolo ritardo, t_{reg}, il tempo per attraversare le porte dello slave. Da lì fino al fronte di salita successivo l'uscita **non cambia**.



Dopo il fronte di salita il registro è stabile per tutto il ciclo

▶ 14 · 04:15 → 08:21 Il latch SR si può anche simulare in Verilog: si scrive con le porte, e le uscite incrociate sono fili. Le forme d'onda del latch SR viste prima sono quelle che dà gtkwave simulandolo.

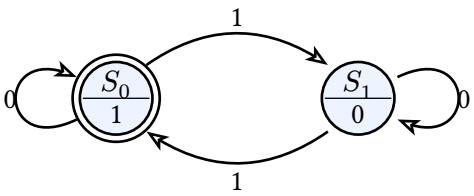
```
module SR(output q, output notq, input s, input r);
  wire w1, w2, nw1, nw2;
  or(w1, r, nw2);   not(nw1, w1);     // NOR in alto
  or(w2, s, nw1);   not(nw2, w2);     // NOR in basso
  assign q = nw1;
  assign notq = nw2;
endmodule
```

2.7 Reti sequenziali

▶ 14 · 17:38 → 20:05

Con i registri una rete può ricordare qualcosa fra un ciclo di clock e l'altro. Ecco un problema dove serve: arriva una sequenza di bit, uno alla volta, e dopo ogni bit si vuole sapere se finora si è visto un numero **pari** di 1 (uscita 1) o dispari (uscita 0). È la **parità** della sequenza. Una rete combinatoria non basta, perché vede solo il bit di adesso. D'altra parte non serve ricordare tutta la sequenza: basta sapere se finora gli 1 sono pari o dispari, e ogni bit nuovo aggiorna questa informazione. Quello che la rete deve ricordare del passato è il suo **stato**: qui gli stati sono due, S_0 (pari) e S_1 (dispari).

Gli stati, con le frecce che dicono in che stato si va a seconda dell'ingresso, formano un **automa**. Una **rete sequenziale** è una rete che realizza un automa. In un **automa di Moore** l'uscita dipende **solo dallo stato**, e si scrive dentro lo stato.



Automa di Moore: nello stato sopra il nome, sotto l'uscita. S_0 (doppio cerchio) è lo stato iniziale

- S_0 = pari, uscita 1; S_1 = dispari, uscita 0.
- Un 0 non cambia la parità: si resta nello stesso stato.
- Un 1 la cambia: si passa all'altro stato.

ciclo	**0**	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**
stato	S_0	S_1	S_1	S_0	S_1	S_0	S_0	S_0	S_0	S_1
uscita	**1**	**0**	**0**	**1**	**0**	**1**	**1**	**1**	**1**	**0**
ingresso	1	0	1	1	1	0	0	0	1	–

▶ 14 · 20:06 → 27:19

L'automa si descrive con due funzioni:

- la **funzione di transizione** S' (o σ): dato lo stato e l'ingresso, dà il prossimo stato. Va da stato × ingresso a stato;
- la **funzione delle uscite** Z (o ω): dato lo stato, dà l'uscita.

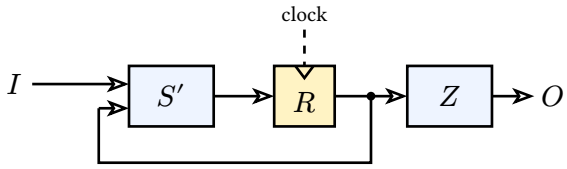
Codificando $S_0 = 0$ e $S_1 = 1$, diventano tabelle di verità, quindi reti combinatorie:

S	I	S'
0	0	0
0	1	**1**
1	0	**1**
1	1	0

$$S' = \overline{S}I + S\overline{I}$$
$$O = \overline{S}$$

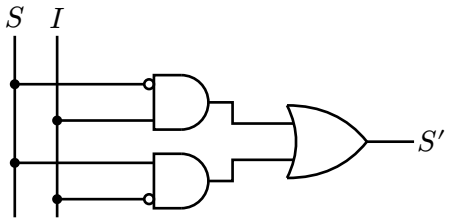
S	O
0	**1**
1	0

Manca un posto dove tenere lo stato: è un **registro**. Lo schema di una rete di Moore è sempre lo stesso: la rete S' calcola il prossimo stato da ingresso e stato attuale, il registro lo memorizza al fronte di clock, e la rete Z calcola l'uscita dal contenuto del registro.



Rete di Moore

Per la parità: S' sono due AND e un OR ($2\Delta t$), il registro ha 1 bit, Z è un NOT (0). Il registro ha WE = 1: a ogni ciclo di clock arriva un ingresso, e lo stato va sempre aggiornato.



La rete S' della parità; il suo ingresso S è l'uscita del registro

▶ 14 · 27:20 → 32:35 A ogni ciclo di clock si presenta un bit della sequenza. All'inizio del ciclo il registro ha già il suo valore e l'ingresso I resta fermo per tutto il ciclo: c'è il tempo di calcolare il nuovo stato, che si scrive nel registro al fronte successivo, e l'uscita.

Il progetto di una rete sequenziale segue cinque passi:

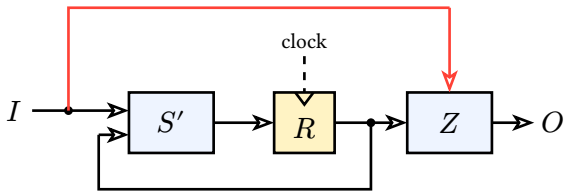
1. automa → 2. reti combinatorie S' e Z → 3. registro di stato bit per tutti gli stati → 4. schema strutturale → 5. ciclo di clock τ

Il registro deve avere abbastanza bit per numerare tutti gli stati: 2 stati, 1 bit; 4 stati, 2 bit. Il **ciclo di clock** τ deve bastare perché si stabilizzino **tutte e due** le reti, più il tempo di scrittura del registro:

$$\tau = \max(T_{S'}, T_Z) + t_{\text{reg}} \qquad \text{parità:} \quad \tau = \max(2\Delta t, 0) + t_{\text{reg}} = 2\Delta t + t_{\text{reg}}$$

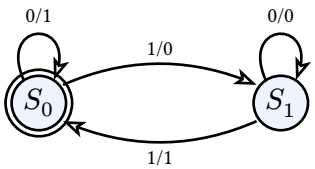
> Con un clock più corto si scriverebbe nel registro un valore non ancora stabile, oppure il registro cambierebbe mentre l'uscita si sta ancora calcolando, e l'uscita non si stabilizzerebbe mai.

▶ 14 · 32:37 → 34:53 In un **automa di Mealy** invece l'uscita dipende dallo stato **e dall'ingresso**: le uscite si scrivono sulle frecce, non negli stati. Lo schema è lo stesso, ma l'ingresso entra anche nella rete Z:



Rete di Mealy: in rosso il collegamento in più

▶ 16 · 01:13 → 07:25 Per esempio la parità con un automa di Mealy. Gli stati restano due, S_0 pari e S_1 dispari, ma l'uscita si scrive sulla freccia, dopo l'ingresso: «1/0» vuol dire «arriva un 1, l'uscita è 0». Da S_0 un 0 lascia la parità pari: si resta in S_0 con uscita 1; un 1 la rende dispari: si va in S_1 con uscita 0. Da S_1 è lo stesso al contrario.



Automa di Mealy per la parità: sulle frecce ingresso/uscita

ciclo	**0**	**1**	**2**	**3**	**4**	**5**	**6**
stato	S_0	S_0	S_1	S_1	S_1	S_0	S_0
ingresso	0	1	0	0	1	0	1
uscita	**1**	**0**	**0**	**0**	**1**	**1**	**0**

Qui l'uscita di ogni ciclo conta già il bit appena arrivato.

Codificando $S_0 = 0$, $S_1 = 1$, una sola tabella dà tutte e due le funzioni, perché ora anche l'uscita dipende da stato e ingresso:

S	I	S'	O
0	0	0	**1**
0	1	**1**	0
1	0	**1**	0
1	1	0	**1**

$$S' = \overline{S}I + S\overline{I}$$
$$O = \overline{S}\,\overline{I} + SI$$

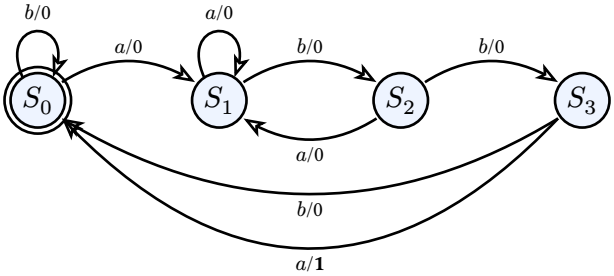
Lo schema è quello di Mealy qui sopra, con un registro da 1 bit.

$$\tau = \max(T_{S'}, T_Z) + t_{\text{reg}} = \max(2\Delta t, 2\Delta t) + t_{\text{reg}} = 2\Delta t + t_{\text{reg}}$$

Gli automi di Mealy di solito sono più compatti: un automa di Moore per lo stesso problema ha lo stesso numero di stati o qualcuno in più.

▶ 16 · 07:31 → 15:00

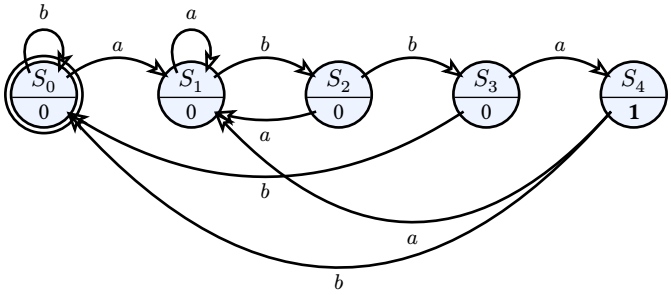
Un uso tipico degli automi è **riconoscere una stringa**: arriva un simbolo per ciclo, e l'uscita vale 1 quando gli ultimi simboli formano la stringa cercata. Esempio: riconoscere *abba* sull'alfabeto $\{a, b\}$, senza sovrapposizioni (l'a finale di un *abba* non conta come inizio del successivo). Ogni stato ricorda quanta parte della stringa si è già vista: S_0 niente, S_1 «a», S_2 «ab», S_3 «abb».



Riconoscitore di *abba* (Mealy)

- Una a quando si aspettava una b non butta via tutto: quella a può essere l'inizio della stringa, quindi si va (o si resta) in S_1.
- In S_3 una b fa *abbb*: si riparte da capo. Una a completa *abba*: uscita 1, l'unica dell'automa, e si torna in S_0.

Con Moore l'uscita sta negli stati, quindi serve uno stato in più: S_4, «ho appena visto *abba*», l'unico con uscita 1. Da lì si riparte: una a è l'inizio di una nuova stringa e porta in S_1, una b riporta in S_0.



Riconoscitore di *abba* (Moore): nello stato sopra il nome, sotto l'uscita

Per provare i due automi si possono seguire *abbaabba* (riconosciuta due volte), *abababba* e *abbabab* (una volta ciascuna).

Per la rete di Mealy si codificano gli stati col loro numero in due bit $s_1 s_0$ ($S_2 = 10$), $a = 0$ e $b = 1$ nell'ingresso x:

s_1	s_0	x	s_1'	s_0'	z
0	0	0	0	**1**	0
0	0	1	0	0	0
0	1	0	0	**1**	0
0	1	1	**1**	0	0
1	0	0	0	**1**	0
1	0	1	**1**	**1**	0
1	1	0	0	0	**1**
1	1	1	0	0	0

$$z = s_1 s_0 \overline{x}$$
$$s_1' = \overline{s_1} s_0 x + s_1 \overline{s_0} x$$
$$s_0' = s_1 \overline{s_0} + \overline{s_1}\,\overline{x}$$

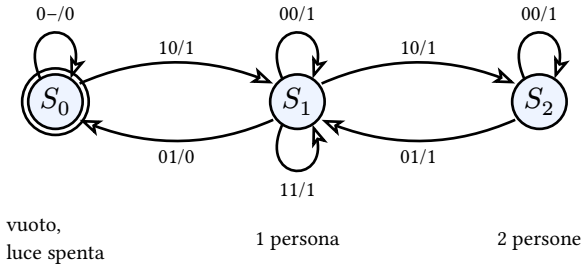
z ha un solo 1: un solo AND, niente OR. In s_1' i due 1 non si raccolgono. In s_0' ci sono quattro 1: 100 e 101 differiscono solo per x e danno $s_1\overline{s_0}$; 000 e 010 differiscono solo per s_0 e danno $\overline{s_1}\,\overline{x}$.

2.8 Reti sequenziali coi componenti e in Verilog

Una rete sequenziale si può anche costruire direttamente con i componenti già visti, come si è fatto per le reti combinatorie con l'approccio strutturale.

▶ 16 · 15:23 → 29:46

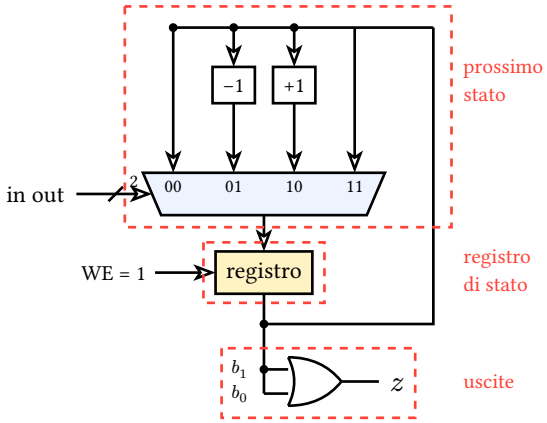
Esempio: uno stanzino con una sola porta, da cui passa una persona alla volta, con posto per due persone. Due ingressi: **in** vale 1 se entra una persona, **out** vale 1 se ne esce una. L'uscita z comanda la lampadina: 1 accesa, 0 spenta. La luce deve restare accesa finché dentro c'è qualcuno. Sulle frecce dell'automa (di Mealy) ci sono i due ingressi **in out**, poi l'uscita; – vuol dire che quel bit non conta.



Lo stanzino

- In S_1 con 11 uno esce e un altro entra: dentro resta una persona.
- Mancano i casi che non possono capitare: da S_0 non può uscire nessuno, in S_2 non può entrare nessuno.

Quello che l'automa ricorda è **quante persone ci sono**: è un contatore. Allora si può costruire così:



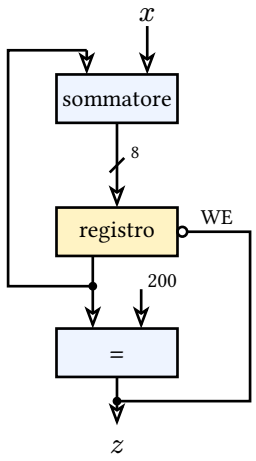
Lo stanzino coi componenti. In rosso le tre parti di una rete sequenziale

- Il registro ha 2 bit, quanti ne servono per contare 0, 1, 2.
- Un MUX a 4 ingressi, comandato da **in out**, sceglie il prossimo valore: 00 e 11 lo lasciano com'è, 01 toglie 1, 10 aggiunge 1. $+1$ e -1 sono gli `x++` e `x--` fatti con un sommatore.
- La luce è accesa se il contatore è 1 o 2, cioè se almeno un bit vale 1: $z = b_1 + b_0$, una OR.

Funziona perché i casi impossibili non capitano: -1 arriva solo se qualcuno esce, quindi il contatore non va sotto 0, e $+1$ non lo porta oltre 2. È un modo «algoritmico» di fare una rete sequenziale: si pensa a cosa va ricordato e a come cambia, e si collegano componenti già pronti.

Che rete è, di Mealy o di Moore? Gli ingressi **in out** arrivano solo alla parte che calcola il prossimo stato; l'OR vede solo il registro. L'uscita dipende solo dallo stato: è una rete di **Moore**.

▶ 16 · 29:54 → 39:12 Ultimo esempio: chiudere le porte di un'aula dopo k ingressi. Ogni volta che entra una persona arriva $x = 1$ (le uscite non si contano). L'uscita z vale 0 con le porte aperte e 1 per chiuderle.



Le porte dopo 200 ingressi

- **Stato**: quante persone sono entrate. Per contare fino a k il registro ha $\lceil \log_2 k \rceil$ bit; per $k = 200$, siccome $2^7 = 128 < 200 \leq 256 = 2^8$, ne servono 8.
- **Uscita**: un comparatore fra il registro e la costante 200, cioè fili fissi a 11001000. Dà 1 quando il registro arriva a 200, e le porte si chiudono.
- **Prossimo stato**: si potrebbe usare un MUX comandato da x fra registro + 1 e registro + 0. Più semplice: un sommatore che fa registro + x. Con $x = 1$ aggiunge 1, con $x = 0$ lascia il valore com'è.

Resta un problema: a porte chiuse $x = 1$ non dovrebbe più arrivare, ma se arriva lo stesso il registro passa a 201, il comparatore torna a 0 e le porte si riaprono. Si blocca il registro: il WE non è più 1 fisso, ma $\overline{z}$. Finché $z = 0$ il registro si scrive; quando $z = 1$ resta fermo a 200. Anche questa è una rete di Moore: x entra solo nel sommatore, e z dipende solo dal registro.

▶ 1718 · 09:11 → 17:43 Una rete sequenziale si può provare in Verilog come una combinatoria. Il primo modo, **strutturale**, ricopia lo schema di Moore o di Mealy pezzo per pezzo: le reti S' e Z si scrivono come si è già visto, con una `primitive` o con `assign`; il registro è un modulo nuovo; le frecce dello schema diventano fili (`wire`). Per la parità di Moore, con $S_0 = 0$ e $S_1 = 1$, le due reti combinatorie sono le tabelle di S' e di O viste prima:

```
primitive s1(output z, input s, input x);   // S': prossimo stato
  table
  // s x : z
     0 0 : 0;
     0 1 : 1;
     1 0 : 1;
     1 1 : 0;
  endtable
endprimitive

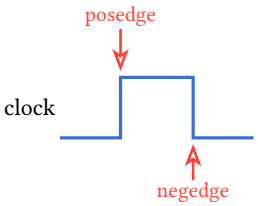
module uscita(output z, input s);           // Z: l'uscita è il NOT dello stato
  assign z = !s;
endmodule
```

▶ 1718 · 17:45 → 24:11 Il registro deve fare quello che fa il registro coi flip-flop: al fronte di salita del clock, se WE = 1, scrive l'ingresso; per tutto il resto del tempo l'uscita è il valore scritto. Ha un parametro N, il numero di bit.

```
module registro #(parameter N = 2)
                 (output [N-1:0] z, input clock, input enable, input [N-1:0] inval);
  reg [N-1:0] stato;            // N flip-flop: qui sta il valore
  initial
    begin
      stato = 0;                // all'accensione vale 0
    end
  always @(posedge clock)       // a ogni fronte di salita del clock...
```

```
    begin
      if (enable)
        stato <= inval;          // ...se WE = 1 scrive l'ingresso
    end
  assign z = stato;              // l'uscita è sempre il contenuto
endmodule
```

- `stato` è un `reg`, come gli ingressi del test: tiene il valore che gli si assegna. Qui lo si assegna solo sul fronte del clock, quindi diventa un registro vero, fatto di flip-flop.
- `initial` funziona come nel test: si esegue una volta, all'accensione.
- `always @(...)` esegue il blocco ogni volta che cambia qualcosa nella lista fra parentesi. `posedge clock` vuol dire «quando il clock passa da 0 a 1» (fronte di salita), `negedge clock` «quando passa da 1 a 0» (fronte di discesa).
- Quando cambia `stato` cambia anche `z`: subito dopo il fronte l'uscita è il valore appena scritto.

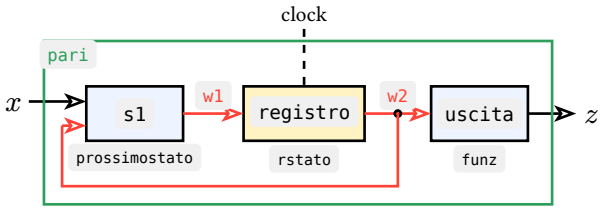


I due fronti del clock

> Nel corso si suppone che registri e memorie, all'accensione, valgano 0. Nei componenti veri serve un segnale apposta; in Verilog lo fa `initial`.

▶ 1718 · 24:11 → 34:23

Ora i tre pezzi si collegano come nello schema di Moore, dentro un modulo `pari` che ha come parametri solo quello che sta fuori: l'ingresso x, il clock e l'uscita z. Il WE non c'è, perché in una rete sequenziale il registro si scrive a ogni ciclo: è fisso a 1, la costante `1'b1`. Dentro servono due fili (in rosso nel disegno): `w1` porta il prossimo stato al registro, `w2` porta il contenuto del registro all'uscita **e** di nuovo al prossimo stato. Un filo che arriva in due posti è sempre lo stesso filo. Sotto ogni blocco c'è il nome della sua copia nel codice.



La parità di Moore in Verilog

```
module pari(output z, input clock, input x);
  wire w1, w2;
  s1 prossimostato(w1, w2, x);
  uscita funz(z, w2);
  registro #(1) rstato(w2, clock, 1'b1, w1);
endmodule
```

Come per il full adder, ogni riga crea una copia di un modulo e la collega, nell'ordine dei parametri: `prossimostato` legge lo stato da `w2` e x e scrive su `w1`; `rstato` è un registro da 1 bit (`#(1)`) che legge `w1` e scrive su `w2`. Per una rete di Mealy l'unica differenza è che anche `uscita` riceve x.

▶ 1718 · 37:20 → 38:24

Per provarla si scrive un test come per le reti combinatorie, con in più il clock. Lo genera un `always` senza lista, che si ripete per sempre:

```
always #1 clock = ~clock;      // ogni unità di tempo si inverte: 0, 1, 0, 1...
```

▶ 1718 · 40:31 → 46:22 È la stessa idea del MUX a quattro fatto con tre MUX a due: si prendono i pezzi e si collegano coi fili. Lì i pezzi erano tutti reti combinatorie, qui due combinatorie e un registro. In Verilog `c[0]` è il bit meno significativo, quello che sceglie dentro le coppie (nella sezione sui componenti si chiamava c_1), e `c[1]` sceglie la coppia.

```
module mux(output z, input x, input y, input c);
  assign z = (!c && x) || (c && y);
endmodule

module mux4(output z, input x0, input x1, input x2, input x3, input [1:0] c);
  wire f1, f2;
  mux A(f1, x0, x1, c[0]);     // sceglie dentro la prima coppia
  mux B(f2, x2, x3, c[0]);     // sceglie dentro la seconda coppia
  mux C(z, f1, f2, c[1]);      // sceglie quale coppia esce
endmodule
```

▶ 1718 · 46:53 → 53:13 Scrivere a mano ogni modulo e ogni filo è lungo. Il secondo modo, **behavioural** (cioè «che descrive il comportamento»), scrive quasi come in C cosa fa la rete, e i componenti li ricava Verilog. Lo schema da descrivere è sempre lo stesso: una formula per S', una per Z, e il registro. Tutto in un modulo:

```
module fsm(output z, input clock, input x);
  reg stato;                    // il registro di stato
  reg nuovostato;               // il prossimo stato, appena calcolato
  initial
    begin
      stato <= 0;
      nuovostato <= 0;
    end
  always @(stato, x)            // ogni volta che cambia lo stato o l'ingresso...
    begin
      nuovostato <= ...;        // ...ricalcola S' (la sua formula)
    end
  always @(posedge clock)       // al fronte di salita il nuovo stato
    stato <= nuovostato;        // entra nel registro
  assign z = ...;               // la formula di Z
endmodule
```

Servono due `reg` perché il prossimo stato si calcola appena cambiano ingresso o stato, ma non deve entrare subito nel registro: solo al fronte di salita del clock. In `initial` basterebbe dare il valore a `stato`; `nuovostato` viene calcolato subito. Con una sola istruzione, come nel secondo `always`, `begin` ed `end` si possono omettere.

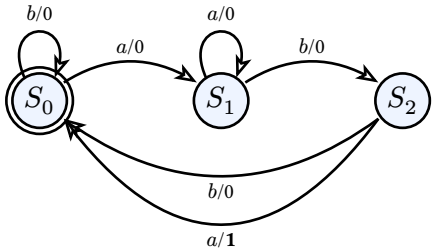
Dentro `initial` e `always` si assegna con `=` o con `<=`. Con `=` gli assegnamenti avvengono uno dopo l'altro, come in C: `a = 0; b = a & c;` dà sempre $b = 0$. Con `<=` avvengono **contemporaneamente**, e ognuno usa i valori di prima: `a <= 0; b <= a & c;` mette in b l'AND fra il vecchio a e c. Per i registri dello stato si usa `<=`: cambiano tutti insieme, come i flip-flop veri.

▶ 1718 · 56:20 → 57:21 Da un modulo scritto così, Quartus (lo strumento di Intel per le FPGA, più grande di Icarus Verilog) ricava da solo la rete di componenti, e sa anche ridisegnare l'automa da cui si era partiti.

▶ 1718 · 57:25 → 58:49 **Due modi per una rete sequenziale in Verilog. Strutturale**: si prende il disegno della rete e lo si ricopia coi moduli, collegati da fili. **Behavioural**: un modulo con il registro di stato e un registro «ombra» per il nuovo stato; un `always @(stato, ingressi)` ricalcola il nuovo stato, un

`always @(posedge clock)` lo copia nello stato, e `assign` calcola le uscite dallo stato e dagli ingressi se la rete è di Mealy, dal solo stato se è di Moore.

Un esempio completo nei due modi: riconoscere aba sull'alfabeto $\{a, b\}$, senza sovrapposizioni. È come $abba$, con uno stato in meno: S_0 non ha visto niente, S_1 ha visto «a», S_2 ha visto «ab».



Riconoscitore di aba (Mealy)

- In S_1 un'altra a può essere l'inizio della stringa: si resta in S_1.
- In S_2 una b fa abb: si riparte da capo. Una a completa aba: uscita 1, e si torna in S_0, perché quell'a non conta come inizio della prossima.

Si codificano gli stati col loro numero ($S_2 = 10$), $a = 0$ e $b = 1$. Lo stato 11 non esiste: le sue righe non contano (–).

s_1	s_0	x	s_1'	s_0'	z
0	0	0	0	**1**	0
0	0	1	0	0	0
0	1	0	0	**1**	0
0	1	1	**1**	0	0
1	0	0	0	0	**1**
1	0	1	0	0	0
1	1	0	–	–	–
1	1	1	–	–	–

$$z = s_1 \overline{s_0}\,\overline{x}$$
$$s_1' = \overline{s_1} s_0 x$$
$$s_0' = \overline{s_1}\,\overline{s_0}\,\overline{x} + \overline{s_1} s_0 \overline{x} = \overline{s_1}\,\overline{x}$$

In s_0' i due 1 differiscono solo per s_0, che quindi non conta: raccogliendo resta $\overline{s_1}\,\overline{x}(\overline{s_0} + s_0) = \overline{s_1}\,\overline{x}$.

Nel modo strutturale ogni formula diventa un `assign`; il filo `toReg` va dal prossimo stato al registro, `fromReg` dal registro alle due reti, tutti e due da 2 bit. È una rete di Mealy, quindi anche `uscite` riceve l'ingresso.

```
module uscite(output z, input in, input [1:0] s);
  assign z = (~in) && s[1] && (~s[0]);
endmodule

module stato(output [1:0] ns, input in, input [1:0] s);
  assign ns[1] = in && (~s[1]) && s[0];
  assign ns[0] = (~s[1]) && (~in);          // la forma semplificata
endmodule

module fsm(output z, input x, input clock);
  wire [1:0] toReg, fromReg;
  registro #(2) rst(fromReg, clock, 1'b1, toReg);
  uscite usc(z, x, fromReg);
  stato sta(toReg, x, fromReg);
endmodule
```

Nel modo behavioural si danno un nome agli stati e ai simboli con `` `define `` (come `#define` in C; il nome si usa con l'apice rovesciato davanti, `` `S1 ``), e la funzione S' si scrive con un `case` sullo stato, che ricopia le frecce dell'automa:

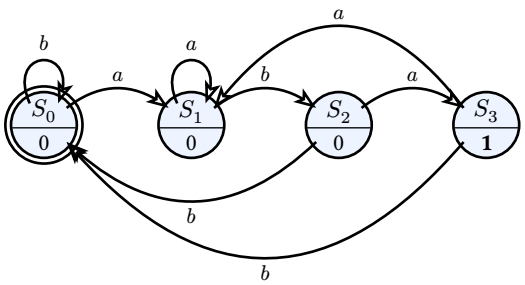
```
`define S0 2'b00
`define S1 2'b01
`define S2 2'b10
`define A 1'b0
`define B 1'b1

module fsm(output z, input x, input clock);
  reg [1:0] stato;
  reg [1:0] nuovostato;
  initial
    begin
      stato = `S0;
    end
  always @(posedge clock)
    stato <= nuovostato;
  always @(stato, x)                          // la funzione S'
    begin
      case (stato)
        `S0: nuovostato <= (x == `A ? `S1 : `S0);
        `S1: nuovostato <= (x == `B ? `S2 : `S1);
        `S2: nuovostato <= (x == `B ? `S0 : `S0);   // comunque in S0
      endcase
    end
  assign z = ((stato == `S2 && x == `A) ? 1 : 0);   // la funzione Z
endmodule
```

I due moduli hanno gli stessi parametri, quindi lo stesso test li prova tutti e due. Qui il clock si inverte ogni 2 unità di tempo: sale a 2, 6, 10… Gli ingressi cambiano fra un fronte e l'altro, così al fronte sono già stabili.

```
module test;
  reg x;                      // un reg per ogni ingresso del modulo da provare
  reg clock;
  wire z;                     // un wire per ogni uscita
  fsm my(z, x, clock);
  initial
    begin
      clock <= 0;
      x <= 0;
    end
  always
    #2 clock = ~clock;
  initial
    begin
      $dumpfile("test.vcd");
      $dumpvars;
      #0 x <= `B;             // la sequenza b a b a a b a
      #3 x <= `A;
      #4 x <= `B;
      #4 x <= `A;
      #8 x <= `B;
      #4 x <= `A;
      #8 $finish;
    end
endmodule
```

Con Moore l'uscita sta negli stati, quindi serve uno stato in più, come per *abba*: S_3, «ho appena visto *aba*», l'unico con uscita 1. Da S_2 una a porta in S_3; da S_3 una a è l'inizio di una nuova stringa e porta in S_1, una b riporta in S_0. Il codice strutturale è lo stesso, cambiano le formule: $S_3 = 11$, quindi l'uscita è `assign z = s[1] && s[0];`, e non riceve più l'ingresso.



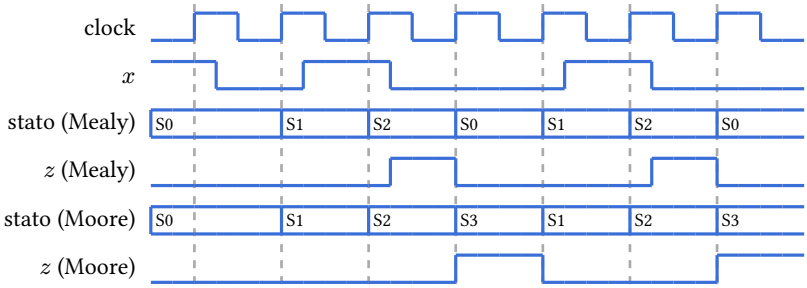
Riconoscitore di *aba* (Moore)

s_1	s_0	x	s_1'	s_0'
0	0	0	0	**1**
0	0	1	0	0
0	1	0	0	**1**
0	1	1	**1**	0
1	0	0	**1**	**1**
1	0	1	0	0
1	1	0	0	**1**
1	1	1	0	0

$$s_1' = \overline{s_1} s_0 x + s_1 \overline{s_0}\,\overline{x} \qquad s_0' = \overline{x}$$

Per s_0' gli 1 sono le quattro righe con $x = 0$: con una a si va sempre in uno stato dispari (S_1 o S_3).

Ecco cosa mostra gtkwave per il test, con la rete di Mealy e con quella di Moore (le linee tratteggiate sono i fronti di salita del clock):



La sequenza *babaaba* ($x = 1$ è b): *aba* viene riconosciuta due volte

Con Mealy z sale appena x diventa a mentre si è in S_2, a metà ciclo, e scende al fronte successivo. Con Moore z sale solo al fronte che porta in S_3, e resta 1 per tutto il ciclo, perché dipende solo dallo stato.